ACCESSABLE CAMPUS: ACCESSIBILITY AUDIT AND BARRIER RESOLUTION SYSTEM

A PROJECT REPORT

Submitted by

YAZHINI PS

(2116240701612)

in partial fulfilment of the End Semester Examination for the course

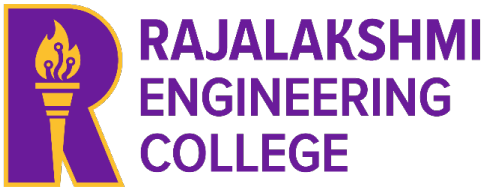
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

November 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “ACCESSABLE CAMPUS: ACCESSIBILITY AUDIT AND BARRIER RESOLUTION SYSTEM” is the bonafide work of **YAZHINI PS** (2116240701612), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE	**SIGNATURE**
Dr. MANORANJINI J HEAD OF THE DEPARTMENT Department of Computer Science and Engineering Rajalakshmi Engineering College Thandalam – 602 105	**Mr. BHUVANESHWARAN B** COURSE FACULTY Department of Computer Science and Engineering Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 11 November 2026.

INTERNAL EXAMINER	EXTERNAL EXAMINER

DECLARATION

I, **YAZHINI PS** (2116240701612), hereby declare that the project report titled "ACCESSABLE CAMPUS: ACCESSIBILITY AUDIT AND BARRIER RESOLUTION SYSTEM", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 11 November 2026

SIGNATURE OF THE STUDENT
YAZHINI PS
2116240701612

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman **Thiru. S. Meganathan**, Chairperson **Dr. (Mrs.) Thangam Meganathan** and Vice-Chairman **Mr. Abhay Meganathan S** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S. N. Murugesan**, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. Manoranjini J**, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **Mr. Bhuvaneshwaran B**, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform, its trial edition and its learning resources, and I thank my family and friends for their unwavering support.

YAZHINI PS

ABSTRACT

Colleges are expected to be open to every student, staff member and visitor, yet physical barriers such as blocked ramps, lifts that are out of service, missing handrails, damaged tactile paving and inaccessible restrooms are common on large campuses. At Rajalakshmi Engineering College such problems are usually reported verbally or on paper, noted in a register or spreadsheet and followed up through repeated personal visits. The process gives no priority, no deadline, no record of who is responsible and no view of which buildings are repeatedly affected, so students with mobility, visual or other needs wait longer than necessary.

This project presents AccessAble Campus, a low-code application built on Zoho Creator to audit campus accessibility and to manage barrier reports from submission to verified closure. Six linked forms — Staff Directory, Campus Building, Campus Facility, Accessibility Audit, Barrier Report and Corrective Action — hold the data, and nine reports (list, kanban, calendar and pivot) and three pages (Command Centre, Staff Workboard and Report and Track) present it. Eight Deluge functions called from six form workflows and one daily schedule calculate an eight-point audit score and a Red/Amber/Green building rating, generate reference numbers, detect recurring barriers, set a service-level target date from the priority, create and route corrective actions to the right maintenance team, escalate overdue work and issue a closure certificate by email. The Zia sentiment analysis service and a keyword-extraction routine enrich each report, and a secured Zoho Cliq connection posts urgent alerts and a daily escalation digest to a channel. Access is organised through an administrator and two permission profiles, Auditor and Facilities Staff, and Community Member.

The application was loaded with 12 buildings, 20 facilities, 18 audits, 20 barrier reports and 13 corrective actions, and its workflows were exercised on this data, including recurring-barrier detection, priority-based target dates, overdue escalation and closure of verified work. The Command Centre shows these figures live from the records. The report documents the design, implementation, testing, security and governance of the system together with its limitations, and shows that a small, well-structured low-code application can bring visibility and accountability to campus accessibility work.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Campus Accessibility*

PROJECT AT A GLANCE

Attribute	Details
Project Title	AccessAble Campus: Accessibility Audit and Barrier Resolution System
Application Domain	Education – Campus Accessibility Auditing and Maintenance Management
Student Name & Register No.	YAZHINI PS – 2116240701612
Zoho Creator Edition	Zoho Creator 15-day free trial
Application Name (Link Name)	AccessAble Campus (accessable-campus)
Published / Live URL	https://creatorapp.zoho.in/yazhini.ps.2024.cse_rajal450/accessable-campus/
Demo Credentials (per role)	Administrator (application owner): yazhini.ps.2024.cse@rajalakshmi.edu.in (Zoho sign-in; password shared with the examiner at the demonstration). Permission profiles for Auditor / Facilities Staff and Community Member are configured under Settings → User Permissions.
Number of Forms / Subforms	6 / 0
Number of Reports (by type)	List: 6; Kanban: 1; Calendar: 1; Pivot: 1; Total: 9
Number of Pages / Dashboards	3 (Command Centre, Staff Workboard, Report and Track)
Number of Workflows	Form: 6; Schedule: 1; Approval: 0; Blueprint: 0; Batch: 0
Deluge Functions / Custom Actions	8 Deluge functions; 0 custom actions
Integrations	Zoho Cliq (OAuth connection, channel #access-alerts); email notifications (sendmail); Zia sentiment analysis (zoho.ai.analyseSentiment)
Roles & Profiles Defined	Administrator (owner); Auditor and Facilities Staff; Community Member. Staff roles held in Staff Directory: Accessibility Administrator, Auditor, Facilities Staff, Verifier
Environments Used	Development (application built and tested in the Development environment)

TABLE OF CONTENTS

ABSTRACT v
PROJECT AT A GLANCE vi
LIST OF TABLES x
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xiii
Chapter 1 INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
Chapter 2 LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
Chapter 3 REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 10
Chapter 4 SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 Data Model 13
4.3 Form Design 15
4.4 Process and Workflow Design 17
4.5 User Interface and Navigation Design 18
4.6 Access Control Design 19
Chapter 5 IMPLEMENTATION 20
5.1 Application Setup 20
5.2 Forms and Data Capture 21
5.3 Reports 23
5.4 Pages and Dashboards 27

5.5 Deluge Scripting....28
5.6 Workflows and Automation....32
5.7 Integrations....34
5.8 User Management and Role-Based Access....36
5.9 AI and Analytics Features....36
5.10 Pro-Code Extension....37
5.11 Mobile and Portal Access....37
Chapter 6 TESTING AND DEBUGGING....39
6.1 Testing Strategy....39
6.2 Test Cases....39
6.3 Debugging Deluge Scripts....41
6.4 Defect Log....42
6.5 User Acceptance Testing....42
Chapter 7 DEPLOYMENT, SECURITY AND GOVERNANCE....43
7.1 Environments and Deployment Strategy....43
7.2 Performance Optimisation....44
7.3 Security Features....44
7.4 Governance and Administration....45
Chapter 8 RESULTS AND DISCUSSION....47
8.1 Application Output....47
8.2 Achievement of Objectives....49
8.3 Comparison with the Existing System....49
8.4 Limitations of the Low-Code Approach....49
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum....50
Chapter 9 CONCLUSION AND FUTURE ENHANCEMENTS....51
9.1 Conclusion....51
9.2 Future Enhancements....51
9.3 Learning Outcomes and Career Relevance....51
REFERENCES....53
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST....54
APPENDIX B – COURSE OUTCOME MAPPING....56
APPENDIX C – COMPLETE DELUGE SOURCE CODE....57
APPENDIX D – USER MANUAL....79
D.1 Accessing the Application....79
D.2 Instructions for the Community Member....79
D.3 Instructions for the Auditor....79
D.4 Instructions for the Accessibility Administrator....79
D.5 Instructions for Facilities Staff and the Verifier....80

APPENDIX E – EVALUATION RUBRIC.....81
APPENDIX F – VIVA-VOCE RECORD SHEET.....83

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....5
Table 2.2: Summary of Literature Survey....6
Table 3.1: Stakeholders and User Roles....8
Table 3.2: Functional Requirements....8
Table 3.3: Non-Functional Requirements....9
Table 3.4: Hardware and Software Requirements....10
Table 3.5: Use Case Description – Report a Barrier (UC-01)....11
Table 3.6: Use Case Description – Review and Route a Barrier (UC-03)....11
Table 4.1: Entities and Relationships....14
Table 4.2: Field Specification – Barrier_Report Form....15
Table 4.3: Field Specification – Accessibility_Audit Form....16
Table 4.4: Field Specification – Corrective_Action Form....17
Table 4.5: Role–Permission Matrix....19
Table 5.1: Reports Implemented....23
Table 5.2: Deluge Scripts Implemented....28
Table 5.3: Workflows Implemented....32
Table 5.4: Integrations Implemented....34
Table 6.1: Test Cases....39
Table 6.2: Defect Log....42
Table 6.3: User Acceptance Testing Plan....42
Table 7.1: Deployment Checklist....43
Table 7.2: Performance Optimisation Results....44
Table 7.3: Security Controls Implemented....44
Table 8.1: Mapping of Objectives to Outcomes....49
Table 8.2: Existing System versus Proposed Application....49
Table A.1: Feature Coverage Checklist....54
Table B.1: Mapping of Course Outcomes to Report Chapters....56
Table E.1: Evaluation Rubric for the Individual Project....81
Table E.2: Performance Level Descriptors....81
Table F.1: Viva-Voce Questions and Responses....83
Table F.2: Marks Summary....83

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System....4
Figure 3.1: Use Case Diagram of the Application....11
Figure 4.1: System Architecture of the Application....13
Figure 4.2: Entity–Relationship Diagram....14
Figure 4.3: Workflow / Process Flow for the Barrier Report Lifecycle....18
Figure 4.4: Navigation Structure and Wireframe of the Command Centre Page....19
Figure 5.1: AccessAble Campus Application in the Zoho Creator Workspace....20
Figure 5.2: Application Home (Command Centre) and Navigation Sections....21
Figure 5.3: Barrier Report Form in the Form Builder (Building Lookup Field Selected)....22
Figure 5.4: Barrier Report Form in Live Mode....22
Figure 5.5: Accessibility Audit Form in Live Mode....22
Figure 5.6: Schema Builder Showing the Six Related Forms....23
Figure 5.7: All Barrier Reports – List Report....24
Figure 5.8: Barrier Workflow Board – Kanban Report....24
Figure 5.9: All Accessibility Audits – List Report....25
Figure 5.10: All Corrective Actions – List Report....25
Figure 5.11: Building Scorecard – Rating and Photographs....25
Figure 5.12: Building Barrier Pivot Report....26
Figure 5.13: All Campus Facilities – List Report....26
Figure 5.14: Staff Directory – List Report....26
Figure 5.15: Command Centre – Barrier Analytics Charts....27
Figure 5.16: Command Centre – Building Accessibility Scores and Priority Action Queue...27
Figure 5.17: Staff Workboard Page....28
Figure 5.18: Report and Track Page for the Community....28
Figure 5.19: Deluge Functions List of the Application....31
Figure 5.20: processBarrierReport Function in the Deluge Editor....32
Figure 5.21: Form Workflows of the Application (Barrier Report Selected)....33
Figure 5.22: On Barrier Reported Workflow Configuration....33
Figure 5.23: Daily Overdue Escalation Schedule (08:30, Enabled)....34
Figure 5.24: Connection Setup – Zoho Cliq Connection (Connected)....35
Figure 5.25: Alerts and Resolution Messages Posted in the Cliq Channel #access-alerts…….35
Figure 5.26: Daily Escalation Digest Posted in Cliq....36
Figure 5.27: Permission Profiles Configured in Settings → User Permissions....36
Figure 5.28: Barrier Report Detail View Showing AI Sentiment and AI Keywords....37
Figure 5.29: Application in the Phone Layout Preview of Zoho Creator....38
Figure 6.1: Deluge Function Execution Result and Log Message....41
Figure 7.1: Environments Page (Stage and Production Empty)....43

Figure 7.2: Governance Console (Security Policies Not Yet Configured).....45
Figure 7.3: Application Settings with Permissions and Developer Tools.....46
Figure 8.1: Scenario Step 1 – Community Member Opens Report and Track.....47
Figure 8.2: Scenario Step 2 – Barrier Report Stored with Reference, Target Date and AI Fields.....47
Figure 8.3: Scenario Step 3 – Corrective Action Created and Assigned.....48
Figure 8.4: Scenario Step 4 – Completed Action Verified by the Verifier.....48
Figure 8.5: Scenario Step 5 – Barrier Closed with Days to Resolve.....48

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
Deluge	Data Enriched Language for the Universal Grid Environment
ER	Entity–Relationship
FR	Functional Requirement
KPI	Key Performance Indicator
NFR	Non-Functional Requirement
OAuth	Open Authorization
RBAC	Role-Based Access Control
REC	Rajalakshmi Engineering College
SLA	Service-Level Agreement (target resolution time)
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Accessibility in a college campus means that every student, staff member and visitor, including those who use wheelchairs or crutches, have low vision or have other needs, can reach classrooms, laboratories, libraries, cafeterias, restrooms and transport without avoidable obstacles. Ramps, lifts, accessible restrooms, parking bays, pathways, handrails, tactile paving and signage are the facilities that make this possible, and each must be present, usable and kept in working order.

Rajalakshmi Engineering College is a large autonomous campus at Thandalam, Chennai, with blocks such as the Admin Block, Academic Block, C Block, Library, Library Block, Workshop Block, Idea Factory, Tech Lounge, the Hut Cafe and REC Cafe, the BB Court and the college bus bay. The Accessibility Cell has to inspect these facilities periodically, receive complaints from the community, and get the repairs done by the civil maintenance, lift maintenance and transport teams. Today this work depends on conversations, phone calls and paper or spreadsheet records, so nobody has a single, current picture of the condition of the campus.

AccessAble Campus is built to give the Accessibility Cell that picture. It combines structured accessibility audits, community barrier reporting, automatic routing of repairs, verification of completed work and a live dashboard on one Zoho Creator application.

1.2 Problem Statement

Accessibility problems on the campus are reported and handled informally, so they are slow to reach the right team, are not prioritised, are easily forgotten and leave no history. There is no objective measure of how accessible each building is, and the Accessibility Cell cannot tell which facilities fail repeatedly or which complaints have crossed their deadline. The specific pain points are:

- Complaints are made verbally, by phone or on paper and are not recorded in one place, so the reporter receives no reference number or status.
- There is no priority or deadline: a lift that has stopped with students inside is treated like a faded signboard.
- Repair work is forwarded by word of mouth, and completion is not verified, so a barrier may be reported closed while it still exists.
- Repeated failures of the same facility are not noticed, because earlier complaints are not compared with new ones.

- Audits, when done, are not scored, so management cannot compare buildings or plan re-inspections.
- Managers have no real-time dashboard, escalation or alert for overdue work.

1.3 Objectives

- To design a normalised data model and forms for buildings, facilities, accessibility audits, barrier reports, corrective actions and maintenance staff.
- To automate audit scoring, reference-number generation, recurring-barrier detection, target-date calculation, routing of corrective actions, overdue escalation and closure certificates using Deluge scripting and workflows.
- To provide role-based reports, a kanban workflow board, a due-date calendar, a pivot report and dashboard pages (Command Centre, Staff Workboard and Report and Track) for administrators, staff and the community.
- To integrate the application with Zoho Cliq for urgent alerts and daily escalation digests, with email for notifications, and with Zia sentiment analysis for advisory insight on each report.
- To organise access through an administrator and permission profiles for auditors, facilities staff and community members, and to document the testing, deployment and governance of the application.

1.4 Scope of the Project

In scope. The application covers the registration of campus buildings and facilities; the eight-point accessibility audit with an automatic score, result and next-inspection date; the community barrier report with an optional photograph; the review, approval or rejection of reports by the Accessibility Administrator; automatic creation and routing of corrective actions; progress updates by facilities staff; verification of completed work; automatic recurring-barrier detection; overdue escalation; notification by email and Zoho Cliq; and a Command Centre dashboard. The platform is the Zoho Creator web application, which can also be previewed in the phone layout of the Zoho Creator mobile app.

Out of scope. Integration with other Zoho applications such as CRM or Analytics, payment handling, external REST APIs, a public customer portal, Zoho Catalyst extensions, offline mobile use and physical sensors are not part of this work.

1.5 Justification for a Low-Code Approach

The Accessibility Cell is a small team without a software department, and its process changes as audits reveal new kinds of barrier. A low-code platform suits this situation,

because forms, reports, pages, workflows, hosting, user management and mobile access are provided by the platform and the remaining logic can be written as short Deluge functions [1], [2]. Low-code platforms are widely reported to shorten development time and to let domain experts take part in building and maintaining applications, although they also bring limits in customisation and dependence on the vendor [3], [4]. For AccessAble Campus these limits are acceptable: the data model is small, the logic is rule-based, and the built-in role model, audit trail and cloud hosting give the cell security and availability that it could not easily build or maintain itself.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

At present a student or staff member who finds an obstacle, for example a blocked ramp or a lift that has stopped working, tells a staff member, phones the maintenance office or writes a note. The complaint may be written in a register or a spreadsheet, or may not be written at all. The message is passed on to a maintenance team by word of mouth, and the reporter returns repeatedly to ask whether the work has been done. When the repair is finished, nobody checks it and nothing is recorded, so if the same barrier appears again there is no history to show that it has happened before. Accessibility audits, when done, are written as free-text notes and are not scored. Figure 2.1 shows this process.

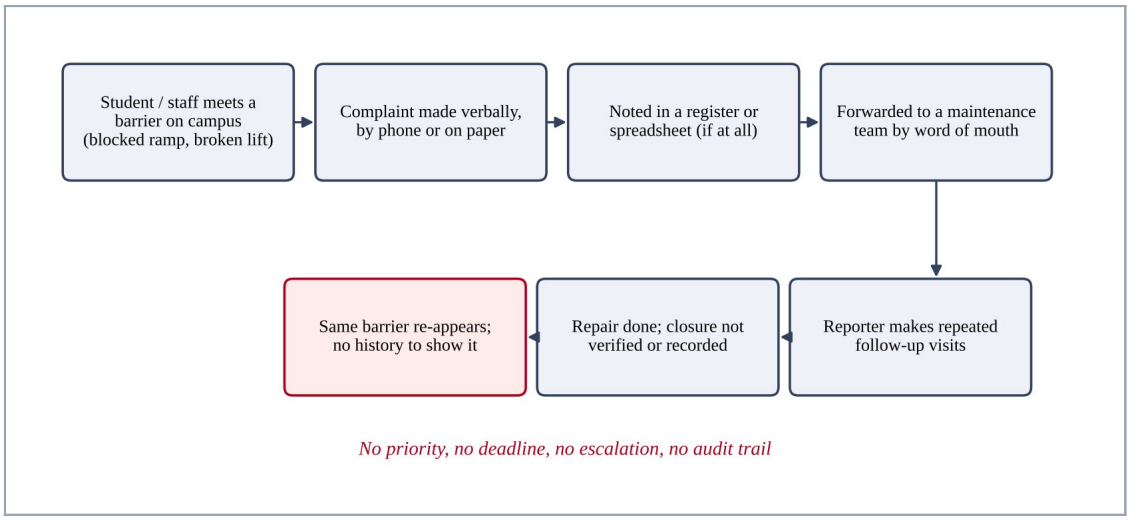


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- No single record: complaints are scattered across registers, spreadsheets and conversations, and the reporter receives no reference number or status.
- No priority, deadline or escalation: urgent barriers wait as long as minor ones, and overdue work is not flagged.
- No verification or history: closed work is not inspected and repeated failures of the same facility are not detected.
- No measurement: audits are not scored, buildings cannot be compared and re-inspections are not scheduled.
- No visibility or alerts: managers cannot see the current position and are not alerted to critical barriers.

2.3 Low-Code versus Traditional Development

Table 2.1 compares the two approaches for this application. The effort figures are the author's own estimates for an application of this size, not measured values.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Estimated 8–10 weeks for database design, back-end, front-end, notification and deployment work	Built in a few days: forms, reports and pages are configured and only 8 Deluge functions are written
Skills required	Back-end and front-end programming, SQL, hosting and DevOps	Data modelling, form and report configuration, and basic Deluge scripting
Hosting & infrastructure	Server, database, backups and SSL to be arranged and maintained	Provided by the Zoho cloud; the application is available at a Creator URL
Mobile application	Separate mobile application or responsive web design	Phone and tablet layouts and the Zoho Creator mobile app are provided by the platform
Security & access control	Authentication, role management and audit logging to be coded	Users, permission profiles and sign-in are built in
Maintenance & change	Code changes, testing and redeployment for every change	New fields, reports and rules are added in the builder and tested immediately
Customisation limits	Unlimited, apart from effort	Bound by platform features, Deluge limits and available page components
Cost	Developer effort and server cost	Zoho Creator subscription (a 15-day free trial was used for this work)

2.4 Literature Survey

Five sources were reviewed: academic studies of low-code development, a comparative study of platforms, and the official Zoho Creator and Deluge documentation. Table 2.2 summarises them and their relevance to this project.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. C. Bock and U. Frank, 2021 [3]	Low-Code Platform, Business & Information Systems Engineering	Defines low-code platforms as environments that support visual, model-driven development of applications by people with limited programming skill	Supports the choice of Zoho Creator for a team without a software department
2	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, 2020 [4]	Supporting the understanding and comparison of low-code development platforms, Proc. 46th Euromicro SEAA	Compares platforms across features such as data modelling, workflow, integration and deployment	Provides the feature dimensions used in the checklist in Appendix A
3	Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, 2021 [5]	Characteristics and challenges of low-code development: the practitioners' perspective, Proc. 15th ACM/IEEE ESEM	Practitioners value speed and ease of use but report limits in customisation, scalability and vendor dependence	Matches the limitations discussed in Section 8.4
4	Zoho Corporation [2]	Zoho Creator Help Documentation	Describes forms, reports, pages, workflows, roles, connections and environments	Primary reference for the configuration described in Chapter 5
5	Zoho Corporation [6]	Deluge Script Reference Guide	Describes the syntax, data-access statements, integration tasks and AI functions of Deluge	Reference for the scripts in Chapter 5 and Appendix C

2.5 Emerging Trends in Low-Code Development

Three trends influenced the design. First, AI-assisted development and AI services inside applications are becoming common; AccessAble Campus uses the Zia sentiment service on each report, but only as advisory information that never changes the priority. Second, citizen development is growing: the Accessibility Cell can maintain the picklists, reports and rules itself because they are configured rather than coded. Third, hyper-automation, in which one event triggers a chain of automatic steps, is used throughout: a submitted report triggers numbering, scoring, alerts and notification without human effort. Composable design also helped: each Deluge function does one job, so processBarrierReport, handleBarrierStatus and refreshBuilding can be reused from several workflows.

2.6 Proposed System

The proposed system is the AccessAble Campus application on Zoho Creator. It removes each limitation in Section 2.2: every complaint is stored with an automatic reference number and status (single record); a service-level target date is set from the priority and an overdue barrier is escalated every morning (priority, deadline and escalation); corrective actions are routed to the right team and closed only after a verifier accepts the work, while a barrier reported three or more times in 60 days is flagged as recurring (verification and history); each audit yields a score, a Green, Amber or Red building rating and a next-inspection date (measurement); and the Command Centre and Zoho Cliq alerts give managers a live view (visibility and alerts).

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Accessibility Administrator	Coordinator of the Accessibility Cell and owner of the application	Manage users and settings, view all data and the Command Centre, review, approve or reject barrier reports, manage buildings, facilities and staff, monitor escalations
Auditor	Inspector from the Accessibility Cell	Submit accessibility audits with the eight-point checklist; view reports
Facilities Staff	Civil, lift and transport maintenance staff	View assigned corrective actions on the Staff Workboard, update progress and the action taken
Verifier	Cell member who inspects completed work	Accept completed work or return it for rework
Community Member	Student, staff member or visitor who meets a barrier	Submit barrier reports with a photograph and track their status on the Report and Track page

In Zoho Creator, the Administrator is the application owner. The Auditor, Facilities Staff and Verifier roles share the permission profile “Auditor and Facilities Staff”, and reporters use the profile “Community Member”; the four staff roles are also recorded in the Staff Directory form so that work can be assigned to them.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow a reporter to submit a barrier report with building, facility, barrier type, description, priority and an optional photograph, with mandatory fields enforced.	Community Member, Auditor	High
FR-02	The system shall generate a unique reference number (ACB-YYYY-NNNN) for each barrier report and a reference (AUD-YYYY-NNNN) for each audit.	System	High
FR-03	The system shall calculate an audit score from eight checklist items, derive an overall result and set the next inspection date.	Auditor	High

FR ID	Requirement	Role(s)	Priority
FR-04	The system shall calculate a Green / Amber / Red rating and open-barrier count for each building from the latest audits.	System	High
FR-05	The system shall detect a recurring barrier (three or more similar reports in 60 days) and raise its priority.	System	Medium
FR-06	The system shall set the target date of a report from its priority (Critical 2, High 5, Medium 10, Low 20 days).	System	High
FR-07	The system shall analyse the sentiment of each description with Zia and extract accessibility keywords, as advisory information.	System	Medium
FR-08	The system shall send an acknowledgement email to the reporter and post Critical, High and recurring reports to the Zoho Cliq channel.	System	High
FR-09	The system shall allow the Administrator to approve or reject a report; approval shall create a corrective action routed to the appropriate team.	Administrator	High
FR-10	The system shall allow facilities staff to update the progress of an assigned action and shall keep the barrier status in step.	Facilities Staff	High
FR-11	The system shall allow a verifier to accept or return completed work; acceptance shall close the barrier, record days to resolve and email a closure certificate.	Verifier	High
FR-12	The system shall escalate overdue barriers every day, mark the actions overdue and post a digest to Zoho Cliq and by email.	System	High
FR-13	The system shall automatically raise barriers for any audit item marked Does Not Meet, without creating duplicates.	System	Medium
FR-14	The system shall provide a Command Centre dashboard with key counts, charts and a priority action queue.	Administrator	Medium
FR-15	The system shall provide a kanban board, a due-date calendar and a building pivot report.	Administrator, Staff	Medium
FR-16	The system shall restrict what each role can see and change through permission profiles.	Administrator	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports shall list their records as soon as they are opened, and form submission shall complete automation (scoring, alerts) without manual steps.

NFR ID	Category	Requirement
NFR-02	Security	Users shall access only the data and components permitted by their permission profile; integrations shall use an OAuth connection and no hard-coded keys.
NFR-03	Usability	The application shall be usable on desktop browsers and in the phone and tablet layouts of Zoho Creator; reporters shall see only the fields they need.
NFR-04	Availability	The application shall be hosted by Zoho Creator’s cloud, with no server to be maintained by the college.
NFR-05	Maintainability	Rules such as target days and rating limits shall be kept in clearly commented Deluge functions, and form fields and picklists shall be changeable in the builder.
NFR-06	Auditability	Record changes shall be traceable through the Creator audit and activity history, and every barrier shall keep its reporter, dates and status.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (15-day free trial edition), India data centre (creatorapp.zoho.in)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
Other Zoho services	Zoho Cliq (alert channel) and Zoho email (notifications); Zia sentiment analysis
Mobile	Zoho Creator mobile app (Android / iOS); phone layout previewed in the Creator application
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases of the four actors. Table 3.5 describes the key use case UC-01 and Table 3.6 describes UC-03, in which the Administrator reviews a report.

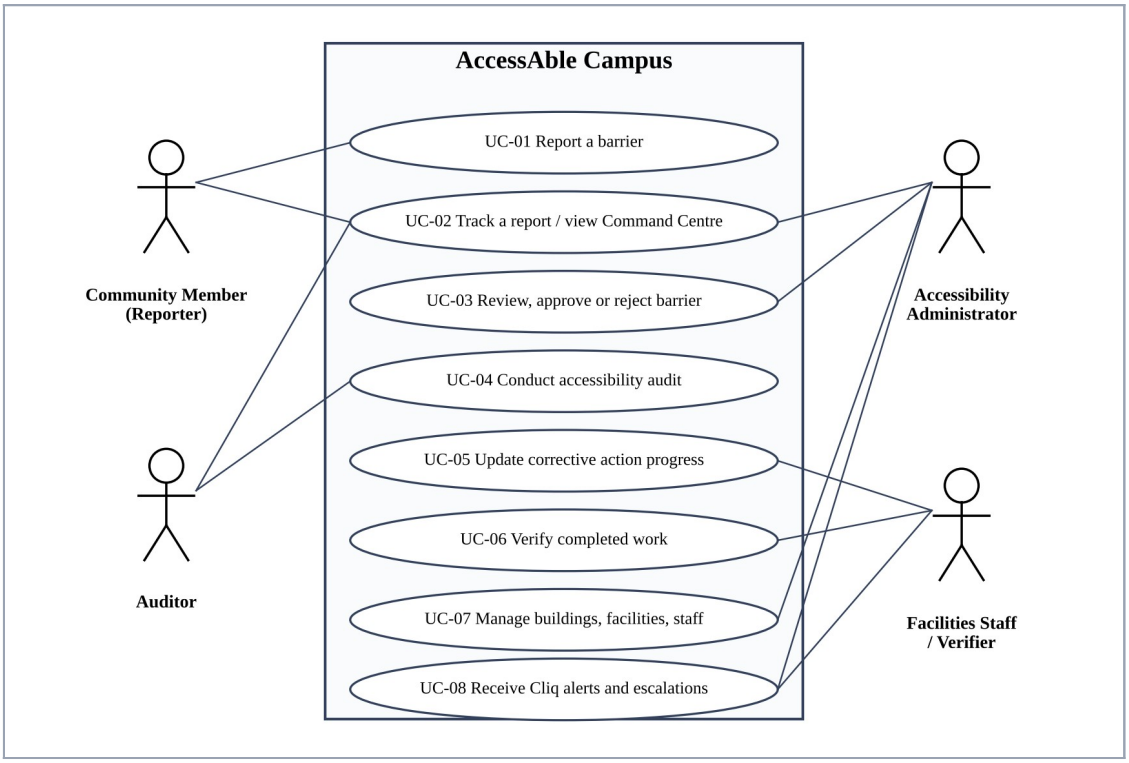


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Report a Barrier (UC-01)

Use Case ID	UC-01
Use Case Name	Report a barrier
Actor(s)	Community Member (reporter); Auditor
Pre-condition	The application page Report and Track or the Barrier Report form is open; buildings and facilities exist in the system
Main Flow	1. The reporter enters name, optional e-mail and reporter type. 2. The reporter selects the building and facility, the barrier type and priority and describes the problem, optionally adding a photograph. 3. The reporter submits the form. 4. The system generates the reference, checks for recurrence, sets the target date, analyses sentiment and keywords, posts a Cliq alert if required and e-mails an acknowledgement.
Alternate Flow	If a mandatory field (building, barrier type or description) is empty, the form shows an error and does not submit. If Zia or Cliq is unavailable, the sentiment is stored as “Unavailable” or the Cliq status as “Failed” and the report is still saved.
Post-condition	A barrier report with a reference number, status Submitted, a target date and AI fields is stored and visible in All Barrier Reports and on the Command Centre
Related FR	FR-01, FR-02, FR-05, FR-06, FR-07, FR-08

Table 3.6: Use Case Description – Review and Route a Barrier (UC-03)

Use Case ID	UC-03

Use Case Name	Review, approve or reject a barrier
Actor(s)	Accessibility Administrator
Pre-condition	A barrier report exists with status Submitted or Under Review
Main Flow	1. The Administrator opens the Barrier Workflow Board or All Barrier Reports. 2. The Administrator checks the report, its priority and AI insights. 3. The Administrator sets the status to Approved. 4. The system creates a corrective action, assigns it to the matching team member, sets the status to Assigned and notifies the staff member by Cliq and e-mail.
Alternate Flow	If the Administrator sets the status to Rejected, the reporter is informed by e-mail and no action is created.
Post-condition	An approved report has one Corrective Action with a target date and a verifier
Related FR	FR-09, FR-10

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows the four layers of the solution. Users reach the application through a web browser, or the phone layout, according to their permission profile. The application layer contains the six forms, nine reports, three pages, eight Deluge functions, six form workflows and one schedule. Data is held in the Creator cloud database, where the forms are related through lookup fields. Integrations are the Zoho Cliq OAuth connection, e-mail sending, the Zia sentiment function and the daily schedule.

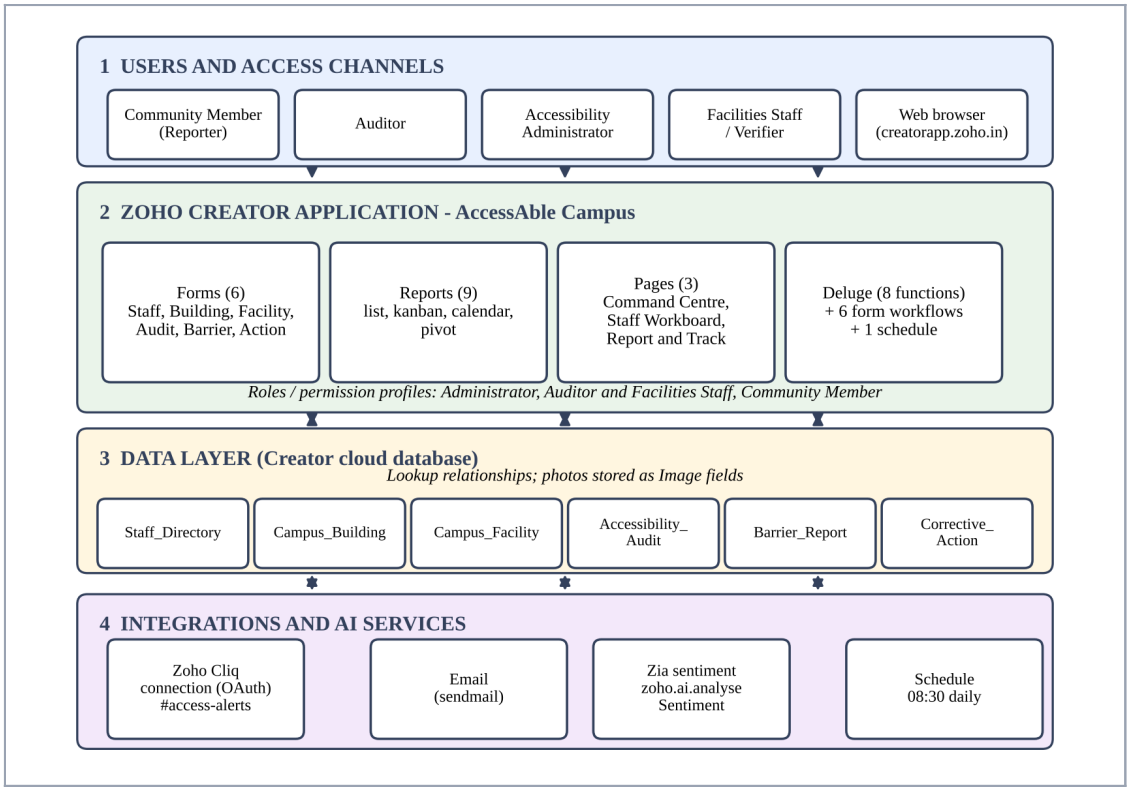


Figure 4.1: System Architecture of the Application

The workflows are thin: each one only calls a function. The functions contain all the logic, which keeps the rules in one place and lets one function be used from several events.

4.2 Data Model

The data model has six forms. Campus_Building is the parent of Campus_Facility; each facility is audited many times in Accessibility_Audit and may have many Barrier_Report records; each approved report has a Corrective_Action. Staff_Directory supplies the auditor, assignee and verifier. All relationships are lookup fields (one-to-

many). No subform is used, because the eight checklist items are fixed and are better stored as drop-down fields that can be scored and reported directly.

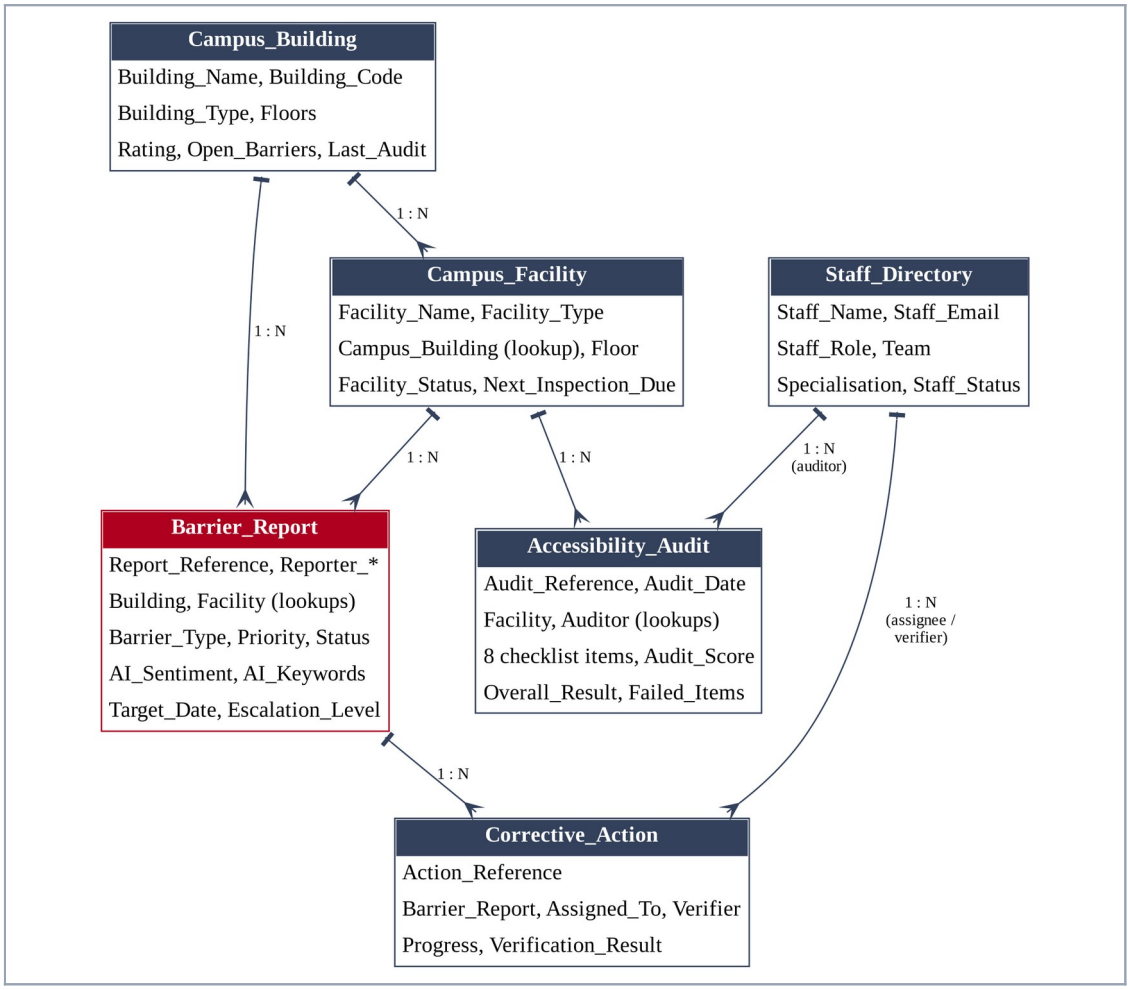


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Campus_Building	Building_Name, Building_Code, Building_Type, Floors, Rating, Open_Barriers	Campus_Facility; Barrier_Report	One-to-Many	Lookup in Campus_Facility and Barrier_Report
Campus_Facility	Facility_Name, Facility_Type, Floor, Facility_Status, Next_Inspection_Due	Accessibility_Audit; Barrier_Report	One-to-Many	Lookup in Accessibility_Audit and Barrier_Report
Accessibility_Audit	Audit_Reference, Audit_Date, eight checklist items, Audit_Score, Overall_Result	Campus_Facility; Staff_Directory	Many-to-One	Lookup fields Facility and Auditor

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Barrier_Report	Report_Reference, Barrier_Type, Priority, Status, Target_Date, AI_Sentiment	Corrective_Action	One-to-Many	Lookup in Corrective_Action
Corrective_Action	Action_Reference, Progress, Verification_Result, Target_Date	Barrier_Report; Staff_Directory	Many-to-One	Lookups Barrier_Report, Assigned_To, Verifier
Staff_Directory	Staff_Name, Staff_Email, Staff_Role, Team, Specialisation, Staff_Status	Accessibility_Audit; Corrective_Action	One-to-Many	Lookups in the two forms

4.3 Form Design

The field specifications of the main forms are given in Tables 4.2 to 4.4. Link names follow the Zoho Creator convention of using the label with underscores. Fields marked system are filled by Deluge and are hidden from the person submitting the form.

Table 4.2: Field Specification – Barrier_Report Form

Field Label	Link Name	Field Type	Required	Validation / Default / Remarks
Reference	Report_Reference	Single Line	System	Generated as ACB-YYYY-NNNN; hidden
Reporter Name	Reporter_Name	Single Line	Yes	
Reporter Email	Reporter_Email	Email	No	Used for acknowledgement and closure mails
Reporter Type	Reporter_Type	Radio	No	Student / Staff / Visitor
Building	Building	Lookup	Yes	Lookup to Campus_Building
Facility	Facility	Lookup	No	Lookup to Campus_Facility
Barrier Type	Barrier_Type	Drop Down	Yes	Blocked Ramp, Lift Out of Service, Missing Handrail, Damaged Tactile Paving, Inaccessible Restroom, Missing Signage, Obstructed Pathway, Uneven Surface, Parking Misuse, Transport Boarding Issue, Other
Description	Description	Multi Line	Yes	Text analysed by Zia
Barrier Photo	Barrier_Photo	Image	No	Evidence photograph
Priority	Priority	Drop Down	No	Default Medium; Critical / High / Medium / Low

Field Label	Link Name	Field Type	Required	Validation / Default / Remarks
Status	Status	Drop Down	System	Submitted → Under Review → Approved → Assigned → Work in Progress → Awaiting Verification → Closed; also Rejected, Rework Required
Source	Source	Drop Down	System	Community Report / Audit Finding
Reported On, Target Date, Closed On	Reported_On, Target_Date, Closed_On	Date	System	Target date from priority
Is Recurring, Recurrence Count	Is_Recurring, Recurrence_Count	Drop Down, Number	System	Yes when three or more similar reports in 60 days
AI Keywords, AI Sentiment	AI_Keywords, AI_Sentiment	Single Line	System	Written by processBarrierReport
Escalation Level, Days To Resolve, Cliq Status	Escalation_Level, Days_To_Resolve, Cliq_Status	Number, Number, Single Line	System	Maintained by escalation and closure functions

Table 4.3: Field Specification – Accessibility_Audit Form

Field Label	Link Name	Field Type	Required	Validation / Default / Remarks
Facility	Facility	Lookup	Yes	Lookup to Campus_Facility
Auditor	Auditor	Lookup	Yes	Lookup to Staff_Directory
Audit Date	Audit_Date	Date	Yes	Default today (form-load workflow)
Ramp, Pathway, Signage, Lift, Restroom, Parking, Handrails, Tactile Paving	Ramp_Access, Pathway_Clear, Signage_Present, Lift_Operational, Restroom_Accessible, Parking_Usable, Handrails_Present, Tactile_Paving	Drop Down (8 fields)	Yes	Meets / Needs Improvement / Does Not Meet / N/A
Observations	Observations	Multi Line	No	Free text
Audit Reference	Audit_Reference	Single Line	System	AUD-YYYY-NNNN; hidden
Audit Score, Failed Items	Audit_Score, Failed_Items	Decimal, Number	System	Meets = 1, Needs Improvement = 0.5, Does Not Meet = 0; N/A excluded

Field Label	Link Name	Field Type	Required	Validation / Default / Remarks
Overall Result, Audit Status	Overall_Result, Audit_Status	Drop Down	System	Accessible ≥ 80, Partially Accessible ≥ 50, else Not Accessible

Table 4.4: Field Specification – Corrective_Action Form

Field Label	Link Name	Field Type	Required	Validation / Default / Remarks
Reference	Action_Reference	Single Line	System	ACT-YYYY-NNNN, derived from the barrier reference
Barrier Report	Barrier_Report	Lookup	Yes	Lookup to Barrier_Report
Assigned To, Verifier	Assigned_To, Verifier	Lookup	Yes	Lookups to Staff_Directory
Assigned On, Target Date, Verified On	Assigned_On, Target_Date, Verified_On	Date	System	Target date copied from the barrier
Work Instructions	Work_Instructions	Multi Line	No	Generated from the barrier
Progress	Progress	Drop Down	Yes	Not Started / In Progress / Completed
Action Taken	Action_Taken	Multi Line	No	Filled by staff
Verification Result, Verification Comments	Verification_Result, Verification_Comments	Drop Down, Multi Line	No	Pending / Accepted / Rework Required
Is Overdue	Is_Overdue	Drop Down	System	Yes when not completed and past the target date

The Campus_Building, Campus_Facility and Staff_Directory forms are simple master forms. Campus_Building stores the name, code, type (Administrative, Academic, Library, Cafeteria, Innovation, Recreation, Sports, Transport), number of floors, an optional photograph, and the calculated Rating, Open_Barriers, Accessibility_Score and Last_Audit_Date. Campus_Facility stores the facility name, type (Ramp, Lift, Restroom, Entrance, Staircase, Parking, Pathway, Signage, Tactile Paving, Bus Boarding), the building lookup, the floor, status and the calculated Next_Inspection_Due. Staff_Directory stores the staff name, e-mail, role, team, specialisation and status.

4.4 Process and Workflow Design

Figure 4.3 shows the barrier lifecycle and the points at which workflows and Deluge functions run. WF-02 runs when a report is created; WF-03 runs when an administrator edits a report; WF-04 runs when staff or the verifier update an action; and SCH-01 runs

every day at 08:30. The audit process runs through WF-01, which calls handleAudit, which in turn calls computeAuditScore and raises barriers for every item marked Does Not Meet.

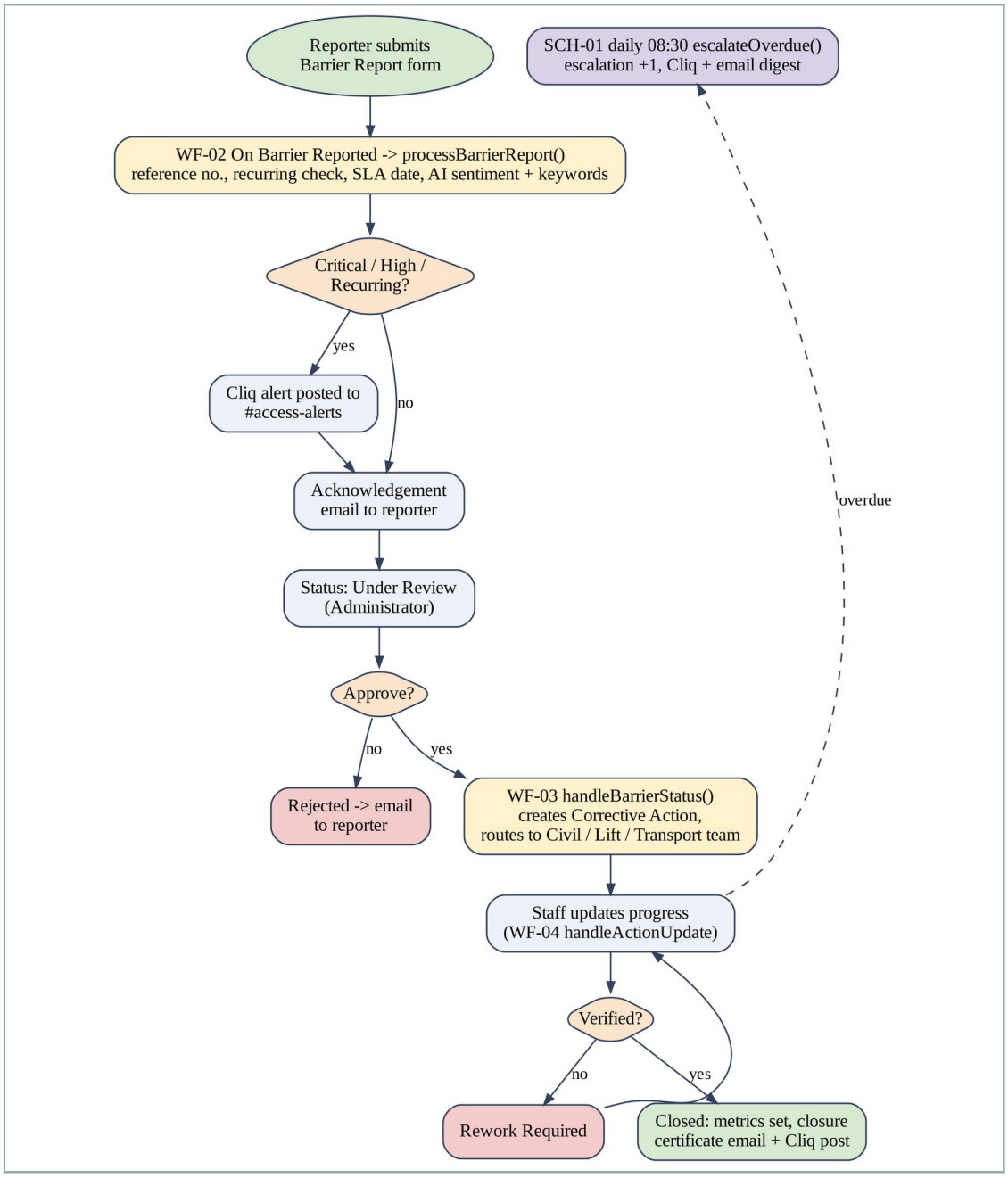


Figure 4.3: Workflow / Process Flow for the Barrier Report Lifecycle

4.5 User Interface and Navigation Design

The left menu is organised in sections that follow the work: Staff Directory, App Preferences, Campus Building, Campus Facilities, Accessibility Audits, Barrier Reports, Corrective Actions, and the three pages Command Centre, Staff Workboard, and Report and Track. Figure 4.4 shows the navigation and the wireframe designed for

the Command Centre page: a banner, a two-row panel of eight key figures, and two chart panels followed by the priority action queue.

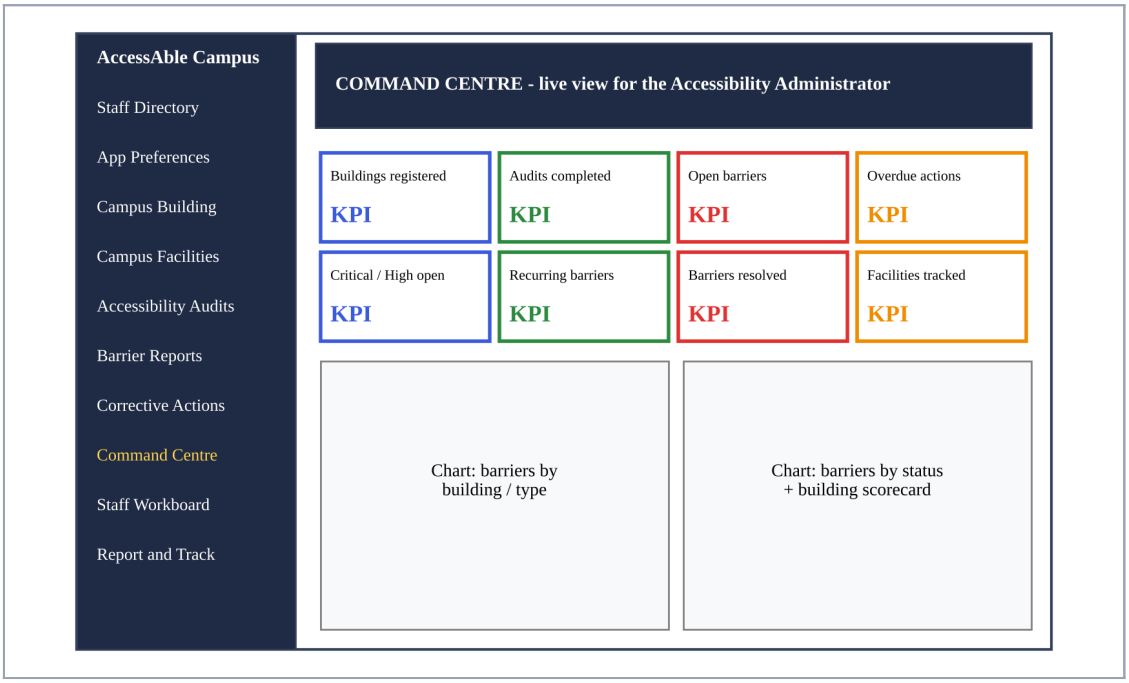


Figure 4.4: Navigation Structure and Wireframe of the Command Centre Page

4.6 Access Control Design

Table 4.5 is the permission matrix designed for the application. A = Add, E = Edit, V = View, D = Delete, — = No access. The Administrator is the owner. The profile “Auditor and Facilities Staff” conducts audits, updates assigned actions and verifies completed work, and the profile “Community Member” can submit barrier reports and view reports and the community page.

Table 4.5: Role–Permission Matrix

Component	Administrator	Auditor and Facilities Staff	Community Member
Form: Staff Directory, Campus Building, Campus Facility	A E V D	V	—
Form: Accessibility Audit	A E V D	A V	—
Form: Barrier Report	A E V D	A V	A V
Form: Corrective Action	A E V D	E V	—
Reports (All Barrier Reports, Audits, Actions, Facilities)	V	V	V (barrier reports)
Page: Command Centre	V	—	—
Page: Staff Workboard	V	V	—
Page: Report and Track	V	V	V

CHAPTER 5
IMPLEMENTATION

This chapter describes how each part of the design was implemented in Zoho Creator, with a screenshot of the live application for each component. All screenshots were taken from the application in the Zoho Creator India data centre on 10 November 2026.

5.1 Application Setup

The application was created in the author's Zoho Creator workspace under the name AccessAble Campus (link name accessable-campus) on 10 November 2026, using the 15-day trial edition. The six forms were created first, followed by the reports, the pages, the Deluge functions and the workflows. The left menu is arranged in sections: Staff Directory, App Preferences, Campus Building, Campus Facilities, Accessibility Audits, Barrier Reports, Corrective Actions, Command Centre, Staff Workboard and Report and Track. A dark navy theme with a single accent colour is used in the menu and on the pages, and the application opens on the Command Centre.

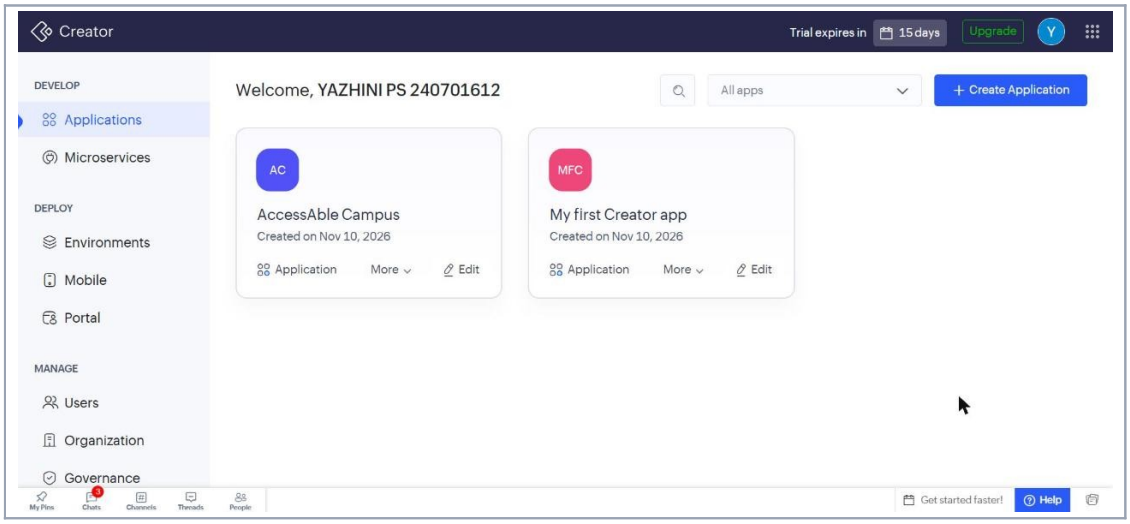


Figure 5.1: AccessAble Campus Application in the Zoho Creator Workspace

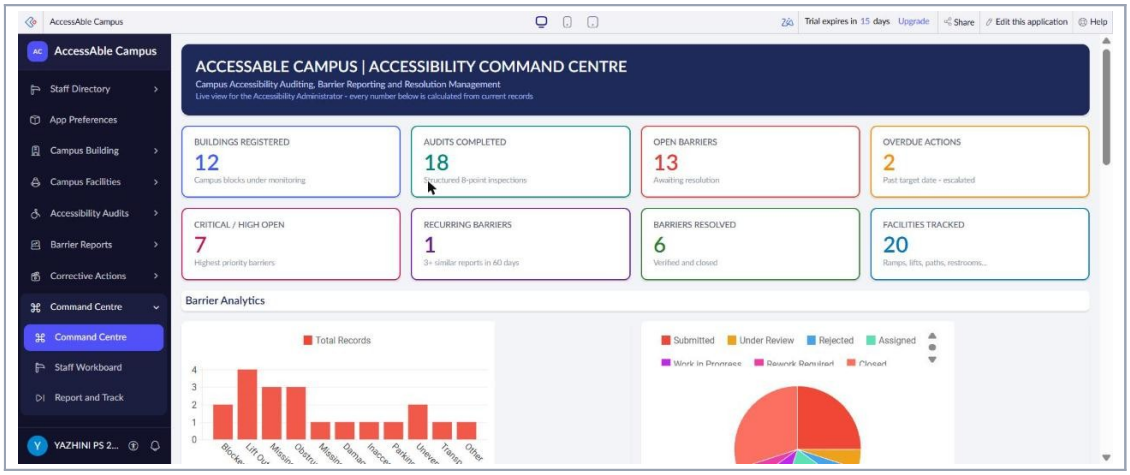


Figure 5.2: Application Home (Command Centre) and Navigation Sections

5.2 Forms and Data Capture

Six forms were built (Section 4.3). The Barrier Report form contains a Name-type text field, an Email, a Radio for the reporter type, two lookup fields (Building and Facility), a drop-down for the barrier type, a multi-line description, an image field for the photograph and a Priority drop-down with Medium as the default. The Accessibility Audit form has two lookups, a date and eight drop-down checklist items. In all, the forms use these field types: single line, multi line, email, phone, number, decimal, date, drop down, radio, lookup, image and auto-generated read-only fields, which is more than eight distinct types.

The lookup from Barrier Report to Campus Building displays the building name and links the two forms; the same technique joins every pair of forms in Figure 4.2. Form behaviour is controlled by two On Load workflows. The Report form hides all system fields (reference, status, source, dates, recurrence, AI fields, escalation, Cliq status and closure data), so that a reporter sees only ten simple fields, while the Audit form defaults the audit date to today and hides the calculated fields. Mandatory fields are marked with an asterisk, and the submit and reset buttons use the default form layout.

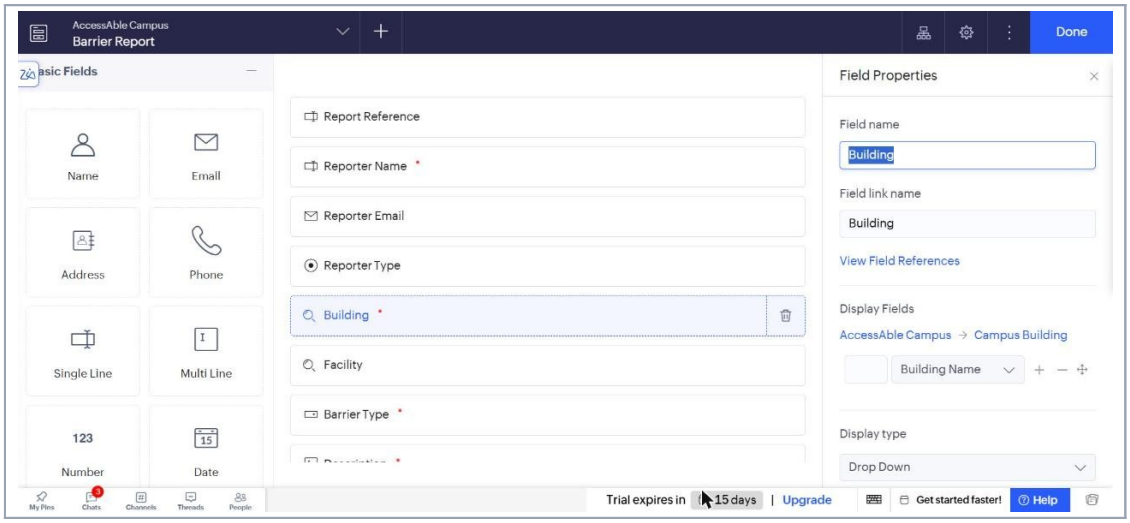


Figure 5.3: Barrier Report Form in the Form Builder (Building Lookup Field Selected)

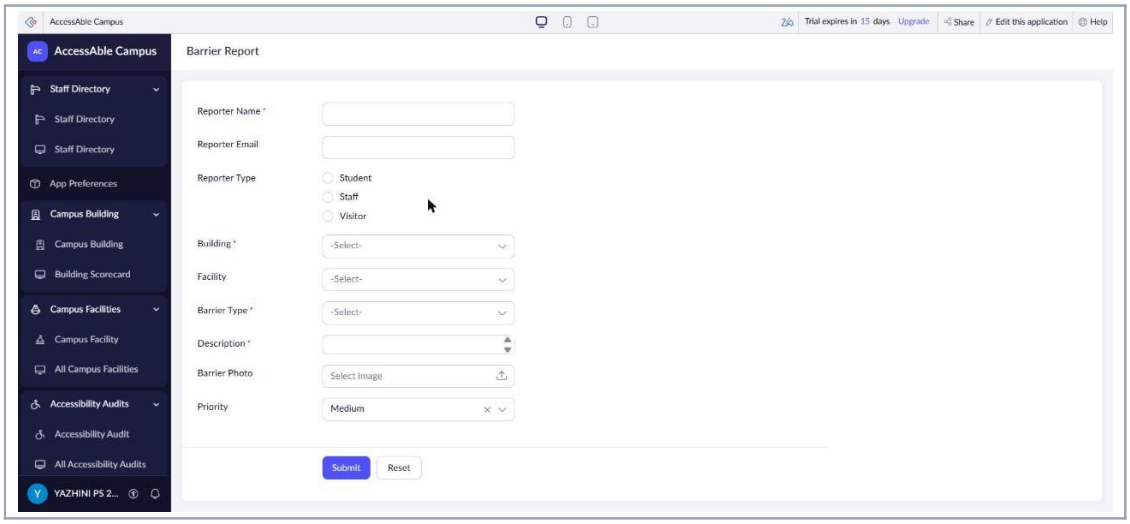


Figure 5.4: Barrier Report Form in Live Mode

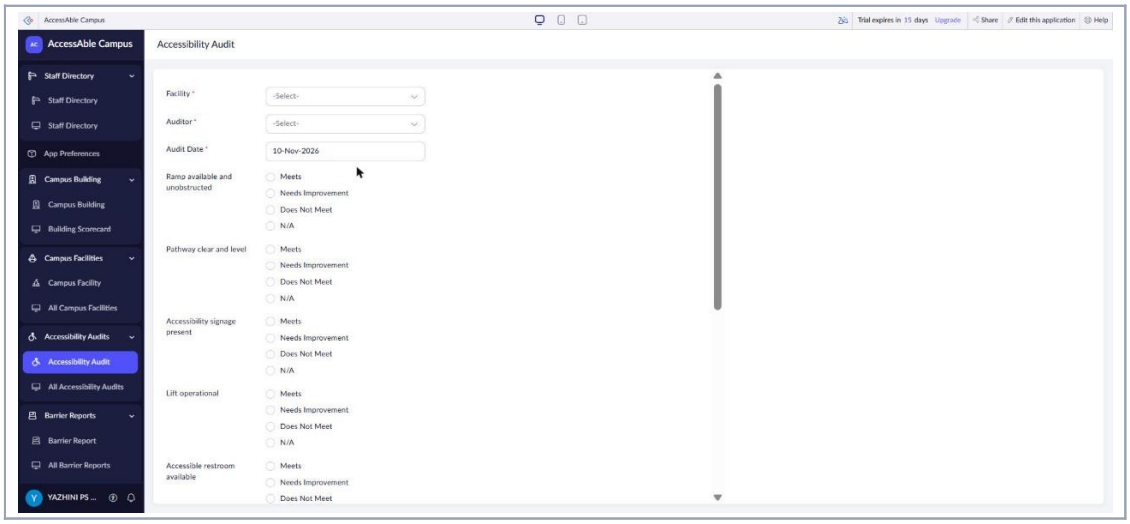


Figure 5.5: Accessibility Audit Form in Live Mode

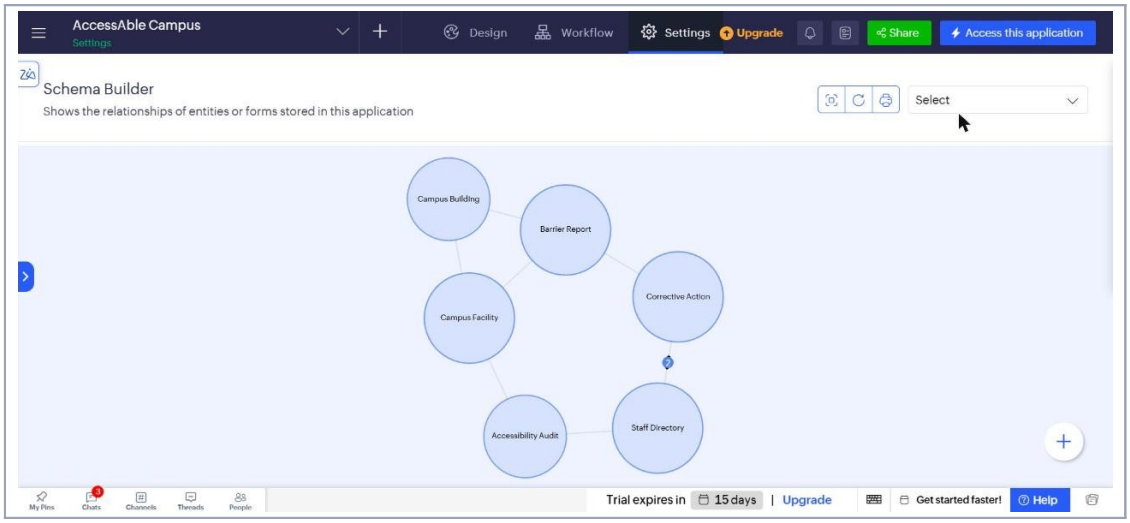


Figure 5.6: Schema Builder Showing the Six Related Forms

5.3 Reports

Nine reports present the data (Table 5.1). The list reports show the main columns of each form and are opened in a quick view; selecting a record opens its detail view. The Barrier Workflow Board is a kanban report whose columns follow the Status field, so that the Administrator sees where every barrier stands. The Action Due Calendar is a calendar report that places each corrective action on its target date, and the Building Barrier Pivot counts open barriers by building and barrier type. The Building Scorecard lists each building with its score, rating, number of open barriers, last audit and photograph, and the Rating column is a Green, Amber or Red label.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Content / Grouping / Sorting	Custom Actions	Roles
All Barrier Reports	List	Barrier_Report	Reference, date, building, facility, type, priority, status, target date, recurrence, escalation, AI sentiment and keywords, reporter; detail view with photograph	None	Admin, Staff, Community
All Accessibility Audits	List	Accessibility_Audit	Reference, date, facility, auditor, score, result, failed items	None	Admin, Staff
All Corrective Actions	List	Corrective_Action	Reference, barrier, assignee, assigned and target dates, progress, overdue flag, verifier	None	Admin, Staff
All Campus Facilities	List	Campus_Facility	Facility, building, type, floor, status, next inspection due, location	None	Admin, Staff
Staff Directory	List	Staff_Directory	Name, role, team, e-mail, phone, specialisation	None	Administrator

Report Name	Type	Source Form	Content / Grouping / Sorting	Custom Actions	Roles
Building Scorecard	List	Campus_Building	Building, code, type, score, rating, open barriers, last audit, floors, photograph	None	Admin, Staff
Barrier Workflow Board	Kanban	Barrier_Report	Columns grouped by Status (Submitted, Under Review, Approved, Assigned, Work in Progress, and so on)	None	Administrator
Action Due Calendar	Calendar	Corrective_Action	Actions plotted by target date	None	Admin, Staff
Building Barrier Pivot	Pivot	Barrier_Report	Rows: building; columns: barrier type; value: number of barriers	None	Administrator

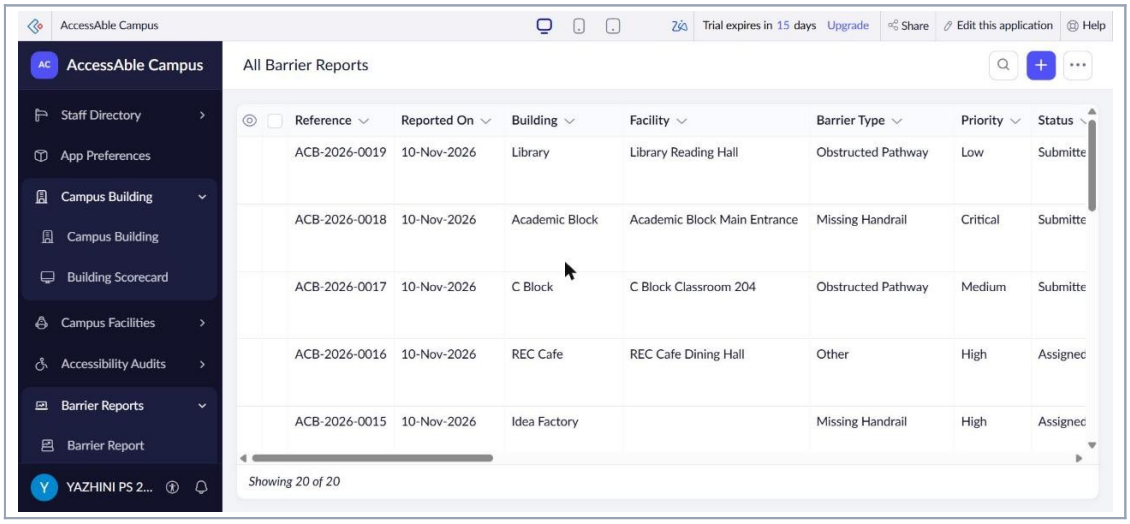


Figure 5.7: All Barrier Reports – List Report

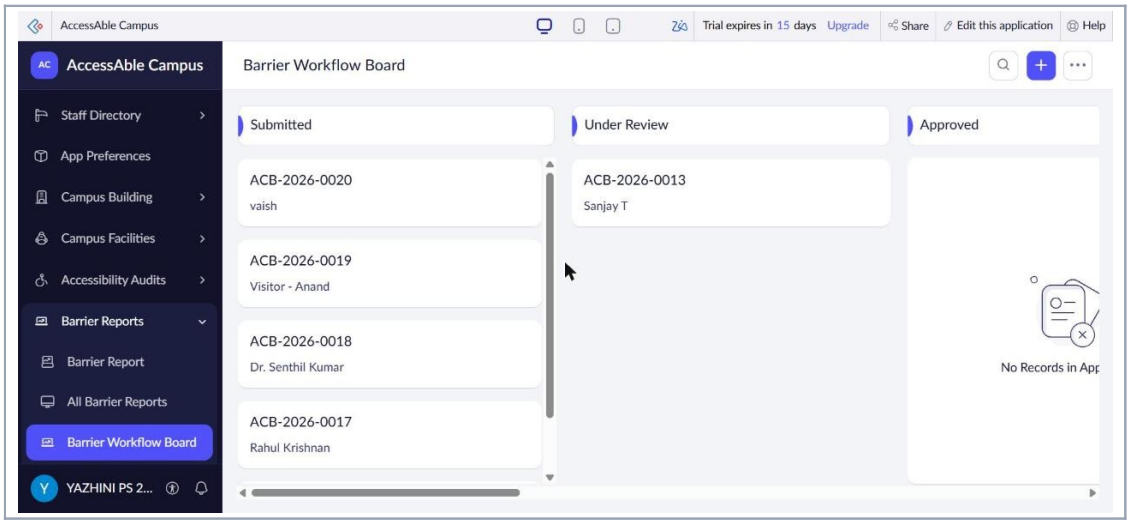


Figure 5.8: Barrier Workflow Board – Kanban Report

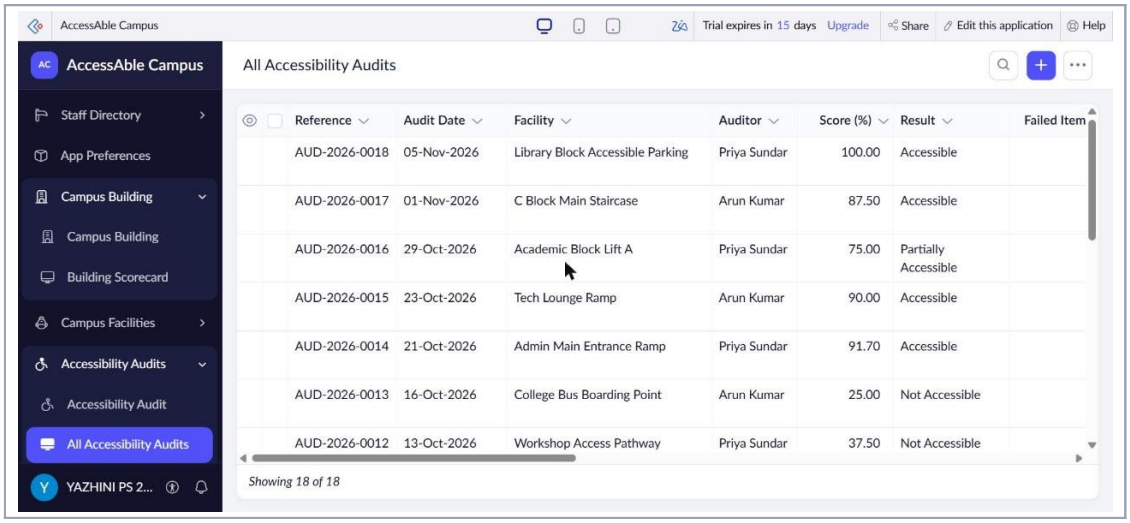


Figure 5.9: All Accessibility Audits – List Report

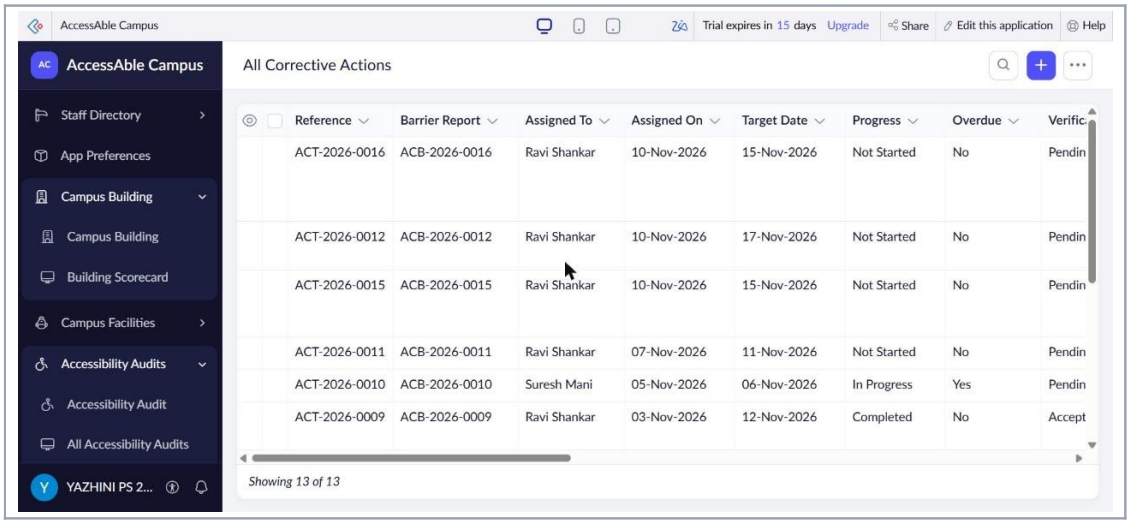


Figure 5.10: All Corrective Actions – List Report

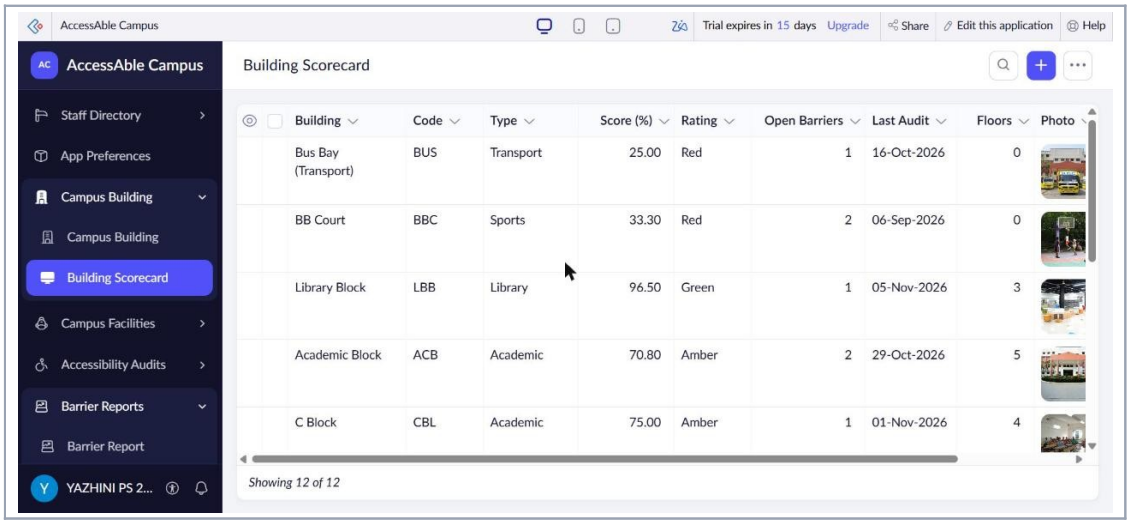


Figure 5.11: Building Scorecard – Rating and Photographs

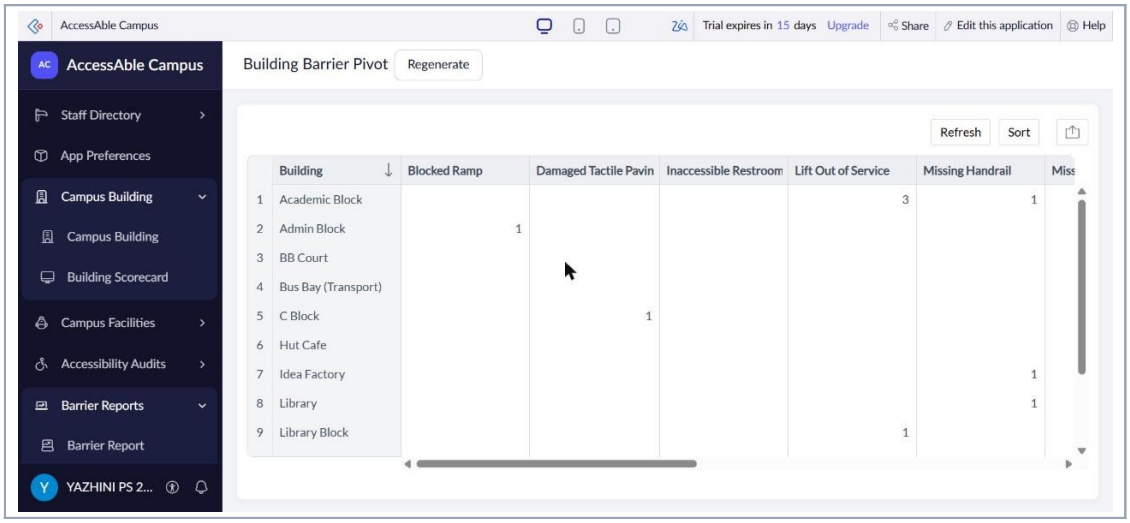


Figure 5.12: Building Barrier Pivot Report

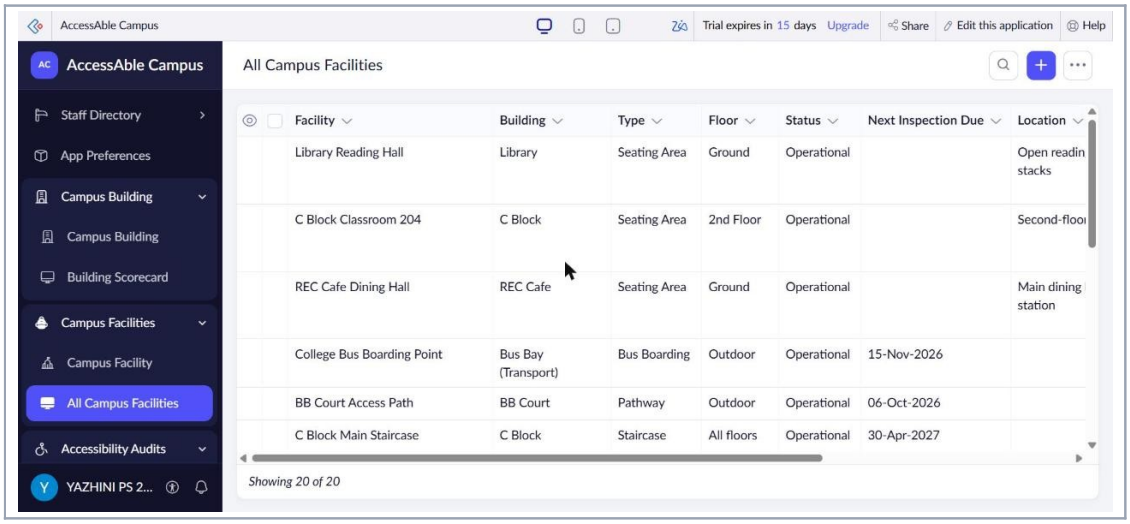


Figure 5.13: All Campus Facilities – List Report

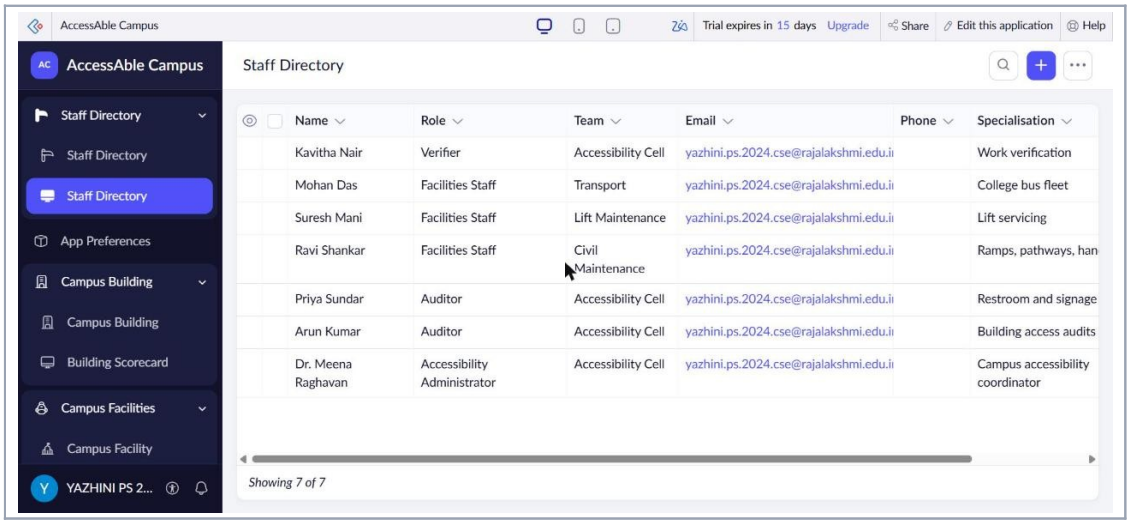


Figure 5.14: Staff Directory – List Report

5.4 Pages and Dashboards

Three pages were built with the Page Builder. **Command Centre** is the dashboard of the Accessibility Administrator. It shows eight key figures (buildings registered, audits completed, open barriers, overdue actions, critical or high open, recurring barriers, barriers resolved and facilities tracked), charts of barriers by priority, status and sentiment, a bar chart of building accessibility scores, an accessibility-result pie chart and the priority action queue, an embedded report of all barrier reports. All numbers are calculated from the current records when the page is opened. **Staff Workboard** is the page of auditors and facilities staff: it shows the counts of actions not started, in progress, awaiting verification and overdue and embeds the Barrier Workflow Board. **Report and Track** is the page for the community: it shows three counters (barriers reported, barriers fixed and being worked on) and embeds the Barrier Report form, so that a reporter can submit a problem from the same page.

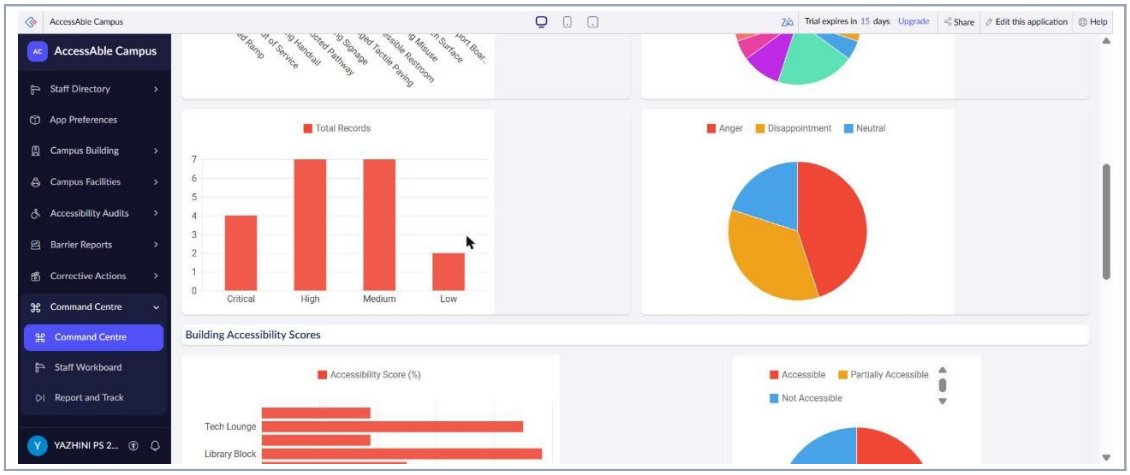


Figure 5.15: Command Centre – Barrier Analytics Charts

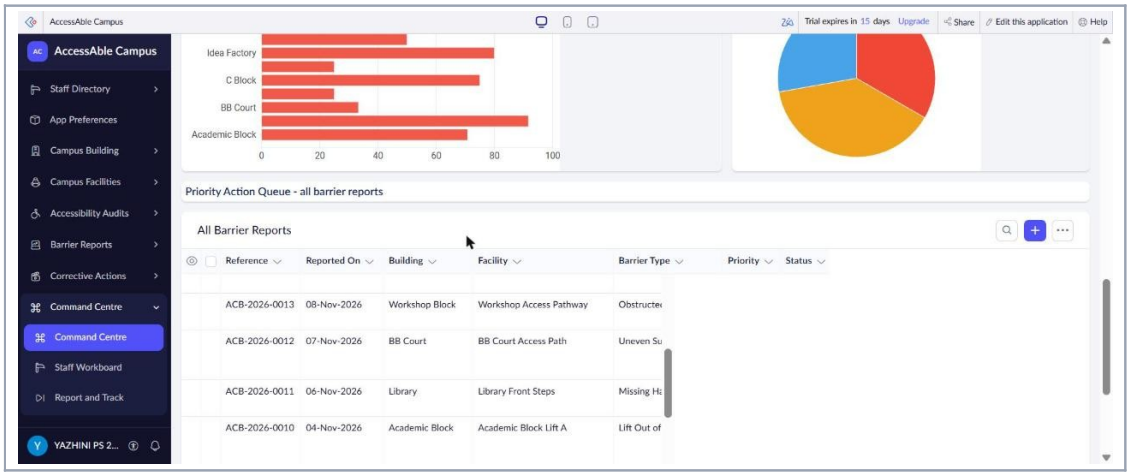


Figure 5.16: Command Centre – Building Accessibility Scores and Priority Action Queue

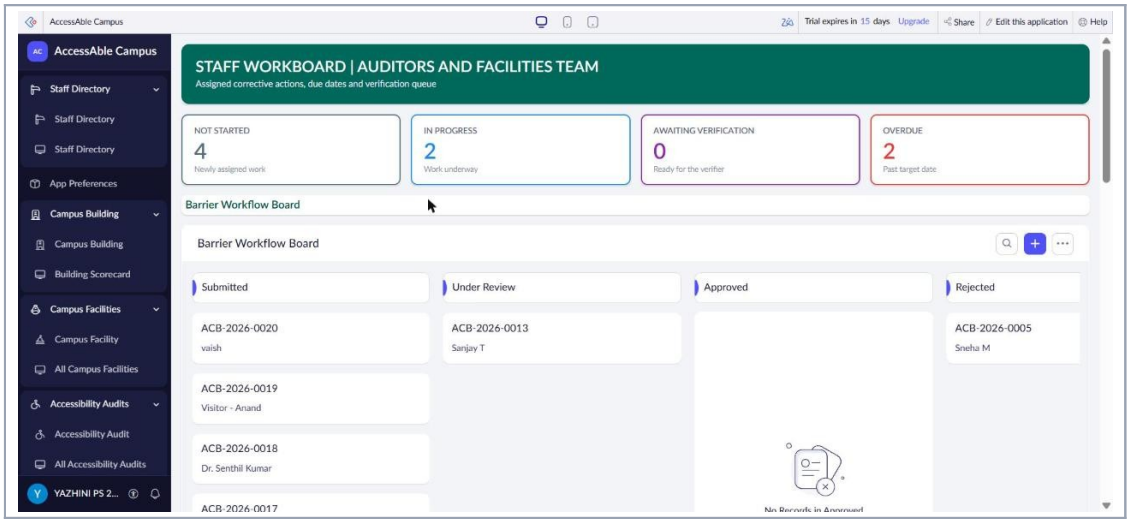


Figure 5.17: Staff Workboard Page

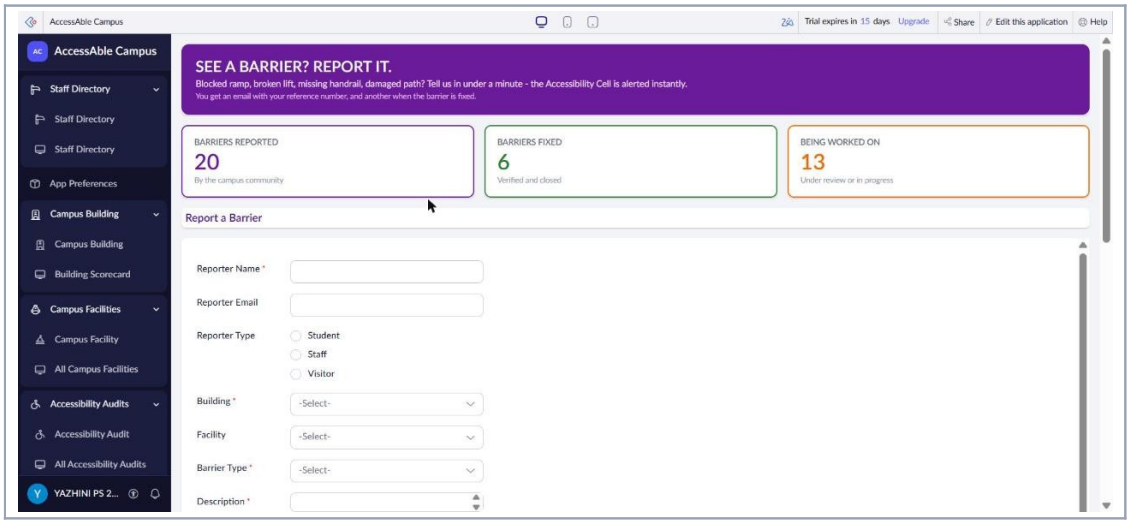


Figure 5.18: Report and Track Page for the Community

5.5 Deluge Scripting

The logic of the application is written in eight Deluge functions and two form-load scripts (Table 5.2). The workflows only call the functions, so each rule lives in one place. The complete source is in Appendix C; the most important parts are explained below. The function list and the editor are shown after Code 5.4.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Function processBarrierReport; called by WF-02 (Barrier Report → On Success of create)	Reference number, recurrence check, SLA target date, Zia sentiment, keyword extraction, Cliq alert, acknowledgement e-mail	fetch with criteria, count, if-else, map, list, for each, try-catch, zoho.ai.analyseSentiment, zoho.cliq.postToChannel, sendmail

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-02	Function handleBarrierStatus; called by WF-02 and WF-03	Create and route the corrective action on approval, inform the reporter on rejection, set closure metrics and certificate on closure	fetch, insert into, range from 1 to 1, string concatenation, daysBetween, sendmail, Cliq post
DS-03	Function handleActionUpdate; called by WF-04	Keep barrier status in step with action progress and verification result	if-else chain, update record, date comparison
DS-04	Function handleAudit; called by WF-01	Raise barriers for each Does Not Meet item without duplicates and alert on poor audits	list of maps, for each, insert into, count
DS-05	Function computeAuditScore; called by DS-04 and DS-08	Score, result, failed items, audit reference and next inspection date	list, for each, round, nested if-else
DS-06	Function refreshBuilding; called by DS-01, DS-02, DS-03, DS-05, DS-07	Open-barrier count, building score, rating and last audit date	aggregate count, sort by … range from 1 to 1
DS-07	Function escalateOverdue; called by SCH-01 (daily 08:30)	Escalate overdue barriers, flag actions overdue, post digest to Cliq and e-mail	criteria fetch, for each, daysBetween, toString, sendmail
DS-08	Function loadSampleData (run manually)	Load 12 buildings, 20 facilities, 18 audits, 15 barriers with their corrective actions and 7 staff if the application is empty	insert into, map, list, subDay, idempotency check
DS-09	WF-05 and WF-06 – On Load of Barrier Report and Accessibility Audit forms	Hide system fields; default audit date to today	hide, input assignment, zoho.currentdate

Code 5.1: Recurring-barrier detection and SLA target date (DS-01)

```
// DS-01 | processBarrierReport | step 2 and 3
similar = Barrier_Report[Facility == r.Facility && Barrier_Type ==
r.Barrier_Type && Reported_On >= today.subDay(60)].count();
r.Recurrence_Count = similar;
if(similar >= 3)
{
    r.Is_Recurring = "Yes";
    if(r.Priority == "Low") { r.Priority = "Medium"; }
    else if(r.Priority == "Medium") { r.Priority = "High"; }
}
slaDays = {"Critical":2,"High":5,"Medium":10,"Low":20};
r.Target_Date = r.Reported_On.addDay(slaDays.get(r.Priority));
```

The script counts the reports for the same facility and barrier type in the last 60 days with one criteria-based count. Three or more reports mark the barrier as recurring and raise its priority by one level, and the target date is then calculated from the final priority using a map of service-level days.

Code 5.2: Zia sentiment with a safe fallback (DS-01)

```
// DS-01 | processBarrierReport | step 4 (advisory only)
desc = ifnull(r.Description,"");
if(desc != "")
{
    try
    {
        sent = if(desc.length() > 190, desc.subString(0,190),
desc);
        ai = zoho.ai.analyseSentiment(sent);
        emo = "";
        if(ai.containKey("data")) { emo =
ai.get("data").get(0).get("Emotion"); }
        else if(ai.containKey("Prediction")) { emo =
ai.get("Prediction"); }
        r.AI_Sentiment = ifnull(emo,"Unavailable");
    }
    catch (e)
    {
        r.AI_Sentiment = "Unavailable";
    }
}
```

The description is cut to 190 characters because longer text was rejected by the service during testing (Defect BUG-02), and a try-catch block makes sure that a failure of the AI service never stops a report from being saved.

Code 5.3: Audit score and building rating (DS-05, DS-06)

```
// DS-05 | computeAuditScore
for each v in vals
{
    if(v == "Meets") { pts = pts + 1; applicable = applicable +
1; }
    else if(v == "Needs Improvement") { pts = pts + 0.5; applicable
= applicable + 1; }
    else if(v == "Does Not Meet") { applicable = applicable + 1;
failed = failed + 1; }
}
score = 0.0;
if(applicable > 0) { score = (pts / applicable * 100).round(1); }
a.Audit_Score = score;
a.Overall_Result = if(score >= 80,"Accessible",if(score >=
50,"Partially Accessible","Not Accessible"));
f.Next_Inspection_Due = a.Audit_Date.addDay(if(score >=
80,180,if(score >= 50,90,30)));

// DS-06 | refreshBuilding
la = Accessibility_Audit[Facility == f.ID] sort by Audit_Date desc
range from 1 to 1;
b.Rating = if(score >= 80,"Green",if(score >= 50,"Amber","Red"));
```

Items marked N/A are excluded from the calculation. A facility that scores 80 or more is due for inspection again after 180 days, a partially accessible one after 90 days and an

inaccessible one after 30 days. The building score is the mean of the latest audit score of each of its facilities.

Code 5.4: Routing a corrective action to the right team (DS-02)

```
// DS-02 | handleBarrierStatus | Approved
team = "Civil Maintenance";
if(r.Barrier_Type == "Lift Out of Service") { team = "Lift
Maintenance"; }
else if(r.Barrier_Type == "Transport Boarding Issue") { team =
"Transport"; }
stf = Staff_Directory[Team == team && Staff_Role == "Facilities
Staff" && Staff_Status == "Active"] range from 1 to 1;
insert into Corrective_Action
[
    Action_Reference = r.Report_Reference.replaceAll("ACB-","ACT-")
    Barrier_Report = rId
    Assigned_To = stf.ID
    Target_Date = r.Target_Date
    Progress = "Not Started"
    Verification_Result = "Pending"
];
r.Status = "Assigned";
```

Approval of a report creates exactly one corrective action (the script first checks that none exists), picks the active facilities staff member of the matching team, copies the target date and sets the report to Assigned. The assignee is told by Cliq and e-mail.

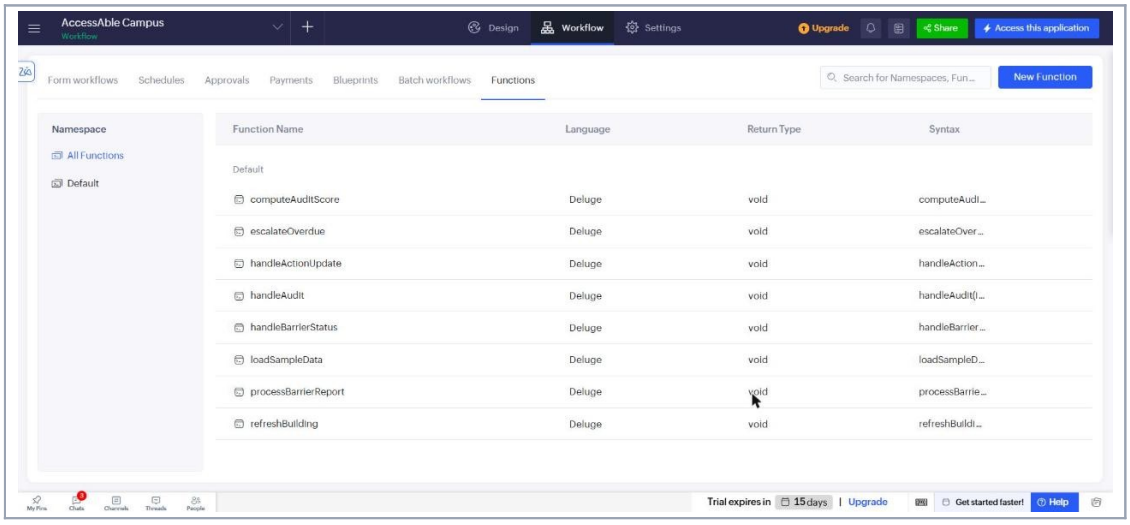


Figure 5.19: Deluge Functions List of the Application

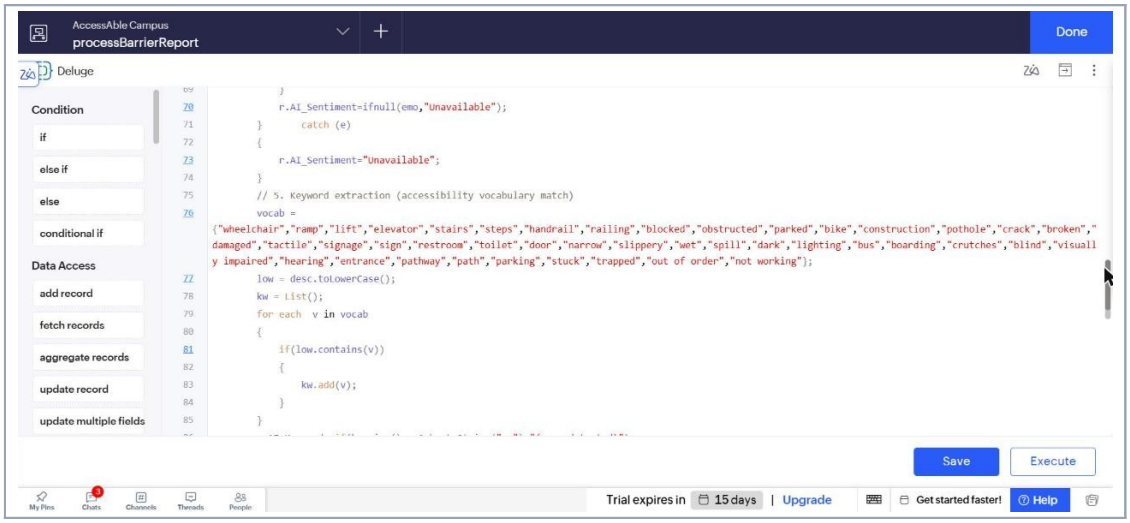


Figure 5.20: processBarrierReport Function in the Deluge Editor

5.6 Workflows and Automation

Six form workflows and one schedule were configured (Table 5.3). No approval process, Blueprint or batch workflow was used: the approval of a barrier is a status change made by the Administrator, and the status transitions are enforced by the functions, which was simpler to maintain for this small process. WF-02 runs after a barrier report is created. It calls processBarrierReport and, for a report that was raised by an audit and is already approved, also calls handleBarrierStatus. WF-03 runs whenever a barrier report is edited. WF-04 runs when a corrective action is created or edited. The schedule SCH-01 calls escalateOverdue every day at 08:30.

Table 5.3: Workflows Implemented

Workflow Name	Form / Event	Type	Condition	Action(s)	Script
WF-01 On Audit Submitted	Accessibility_Audit → Created → On Success	Form	Always	Call handleAudit	DS-04
WF-02 On Barrier Reported	Barrier_Report → Created → On Success	Form	Audit findings already Approved also call handleBarrierStatus	Call processBarrierReport	DS-01, DS-02
WF-03 On Barrier Status Change	Barrier_Report → Edited → On Success	Form	Always	Call handleBarrierStatus	DS-02
WF-04 On Action Progress	Corrective_Action → Created or Edited → On Success	Form	Always	Call handleActionUpdate	DS-03
WF-05 Report Form Load	Barrier_Report → Created → On Load	Form	Always	Hide system fields	DS-09

Workflow Name	Form / Event	Type	Condition	Action(s)	Script
WF-06 Audit Form Load	Accessibility_Audit → Created → On Load	Form	Always	Default date; hide calculated fields	DS-09
SCH-01 Daily Overdue Escalation	Schedule – daily at 08:30	Schedule	Barrier past target date and not closed or rejected	Call escalateOverdue	DS-07

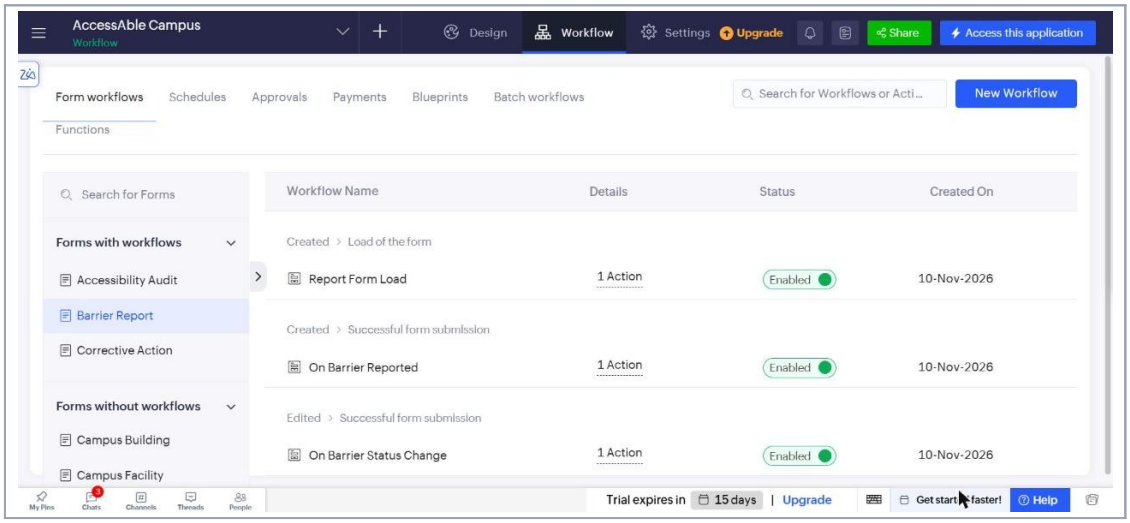


Figure 5.21: Form Workflows of the Application (Barrier Report Selected)

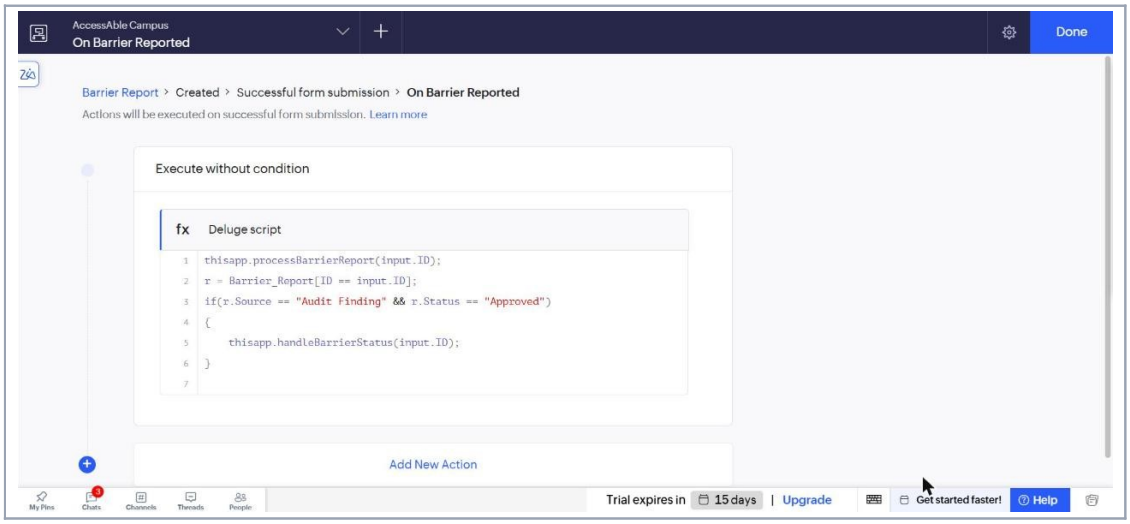


Figure 5.22: On Barrier Reported Workflow Configuration

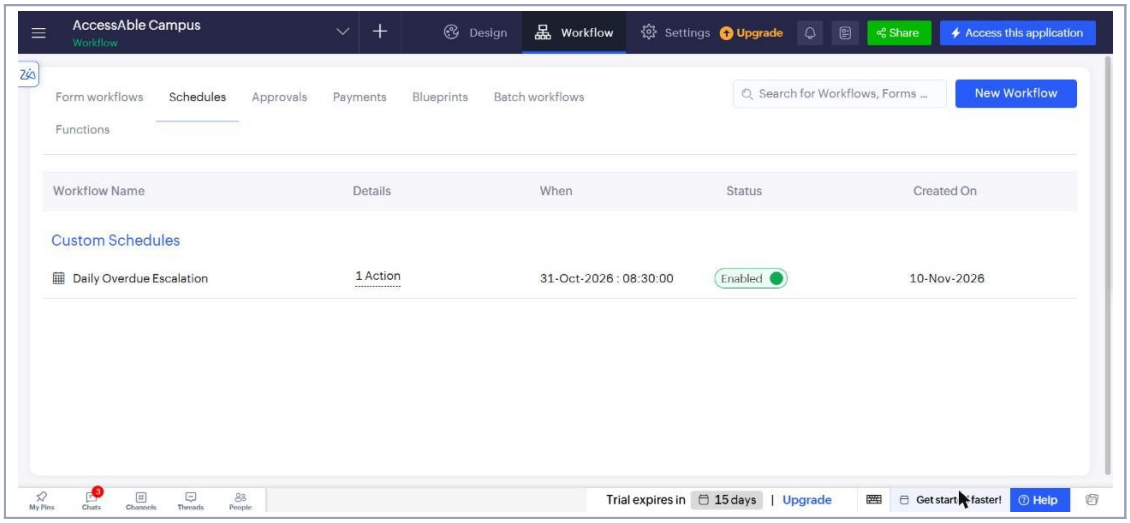


Figure 5.23: Daily Overdue Escalation Schedule (08:30, Enabled)

5.7 Integrations

Three services are used, as listed in Table 5.4. Zoho Cliq is integrated through a connection named zoho_cliq_connection that was authorised once with OAuth 2.0 and is used from Deluge with the zoho.cliq.postToChannel task. Because the connection holds the token, no key or password is written in the code. E-mail is sent with the sendmail task, and the sentiment of each description is obtained from the built-in Zia function. No other Zoho application and no external REST API, webhook or Creator REST API call is used in this application.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Auth.	Data Exchanged / Purpose
Cliq alerts	Zoho Cliq channel access-alerts	Connection zoho_cliq_connection + zoho.cliq.postToChannel	OAuth 2.0	Urgent alerts, assignment, rework, resolution and the daily escalation digest
E-mail	Zoho Creator mail service	sendmail task from the admin user	Platform	Acknowledgement, assignment, verification request, rejection, closure certificate and escalation e-mails
Zia sentiment	Zoho AI	zoho.ai.analyseSentiment	Platform	Emotion label stored in AI_Sentiment

Code 5.5: Posting an alert to Zoho Cliq through the connection (DS-01)

```
msg = "Accessibility barrier " + r.Report_Reference + " | " +
r.Priority + " | " + r.Barrier_Type
      + " at " + bName + " | Due " + r.Target_Date.toString("dd-
MMM-yyyy");
```

```
try
{
    resp = zoho.cliq.postToChannel("accessalerts", msg,
"zoho_cliq_connection");
    r.Cliq_Status = if(resp.get("status") == "OK", "Posted",
"Failed");
}
catch (e)
{
    r.Cliq_Status = "Failed";
}
```

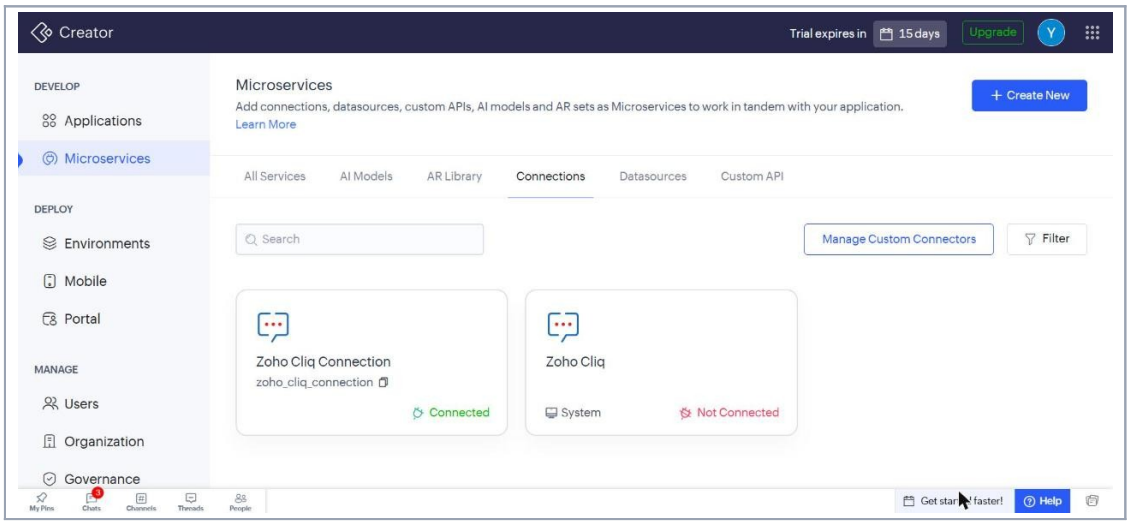


Figure 5.24: Connection Setup – Zoho Cliq Connection (Connected)

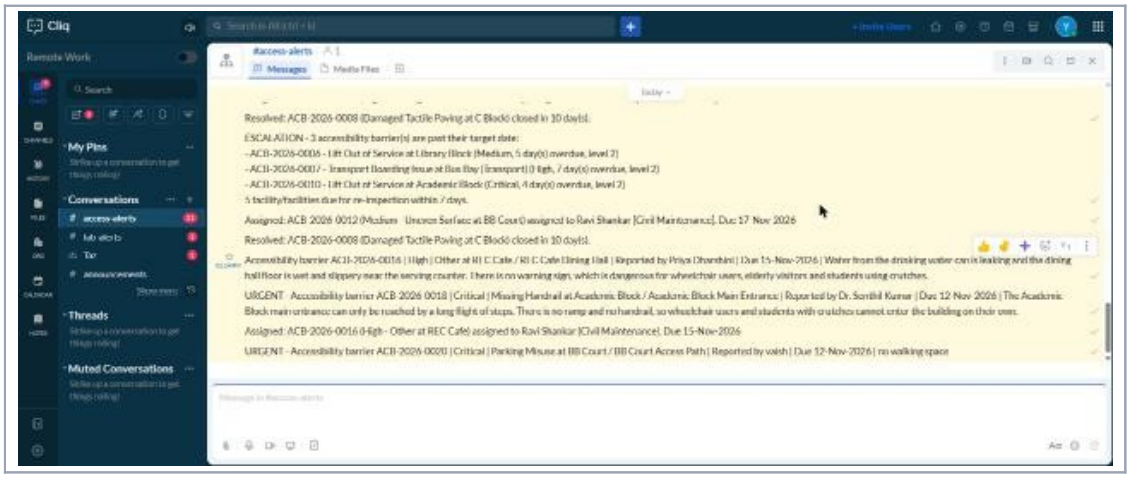


Figure 5.25: Alerts and Resolution Messages Posted in the Cliq Channel #access-alerts

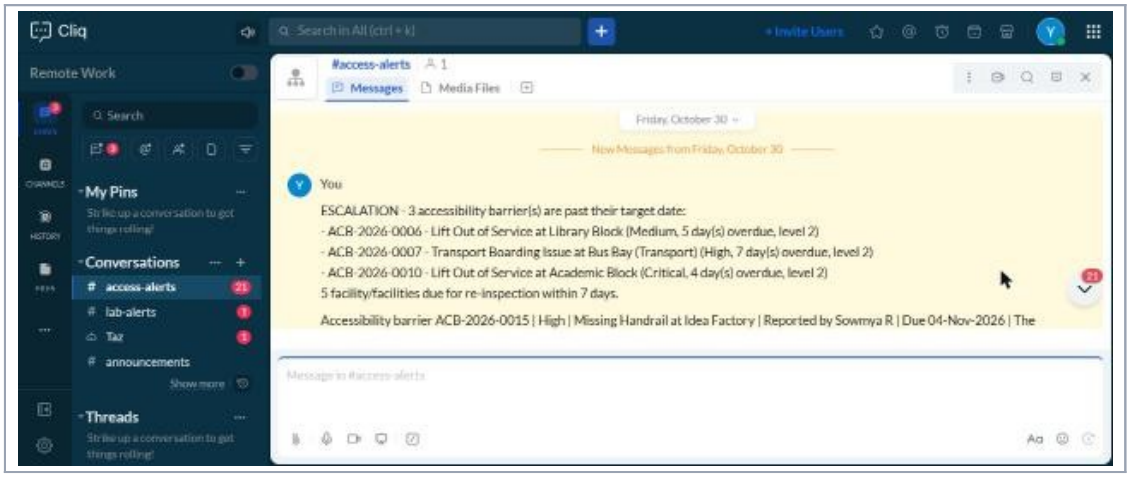


Figure 5.26: Daily Escalation Digest Posted in Cliq

5.8 User Management and Role-Based Access

The application owner is the Accessibility Administrator and has full access. Two permission profiles were defined in Settings → User Permissions, in addition to the default Read and Write profiles: “Auditor and Facilities Staff”, which conducts audits, updates assigned corrective actions and verifies completed work, and “Community Member”, which can submit barrier reports and view reports and the community page only. The reporting side of the system also reduces risk on the form itself, since system fields are hidden from reporters by the On Load workflow. Record-level criteria and a customer portal were not configured in this version, and users are added to the profiles by the Administrator when they are invited to the application.

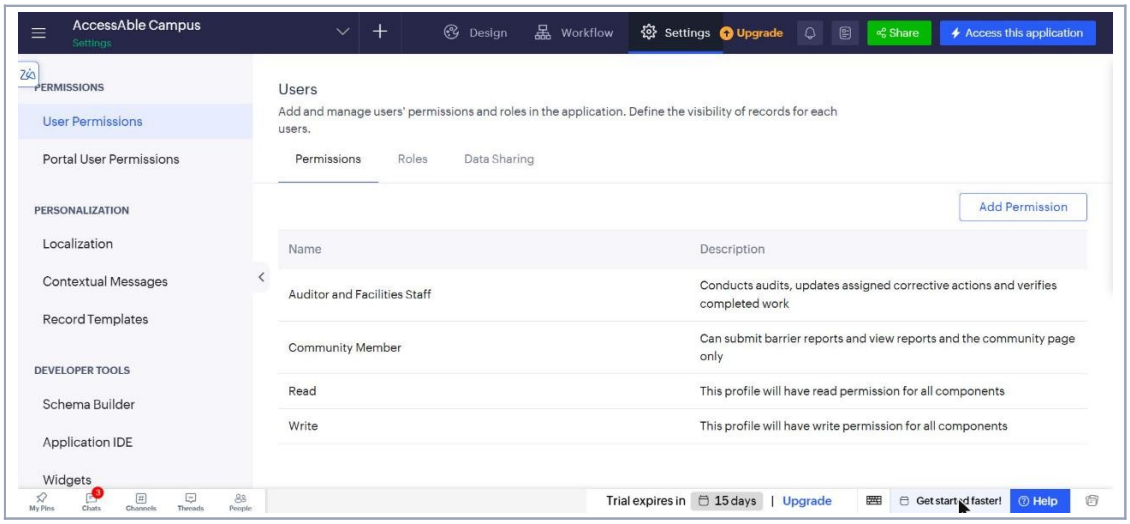


Figure 5.27: Permission Profiles Configured in Settings → User Permissions

5.9 AI and Analytics Features

AI. The Zia sentiment function analyses the description of every new barrier report and the emotion (Anger, Disappointment or Neutral) is stored in AI_Sentiment, so the Administrator can see how strongly a reporter feels about a problem. A vocabulary

routine finds accessibility words, such as wheelchair, ramp, lift, handrail or crutches, in the description and stores them in AI_Keywords. These two outputs are advisory only: they never change the priority, which is decided by the reporter and the recurrence rule. The AI output was checked by comparing the stored emotion and keywords with the description of each report: the keywords of the most recent reports contain the accessibility words found in their text (for example wheelchair and visually impaired), and the emotion is shown beside them in the report. The sample records created by the data loader carry preset values for these two fields.

Analytics. The Command Centre includes bar and pie charts of the barriers by priority, status and sentiment, the building scores, and a pivot report of barriers by building and type. Zoho Analytics was not integrated.

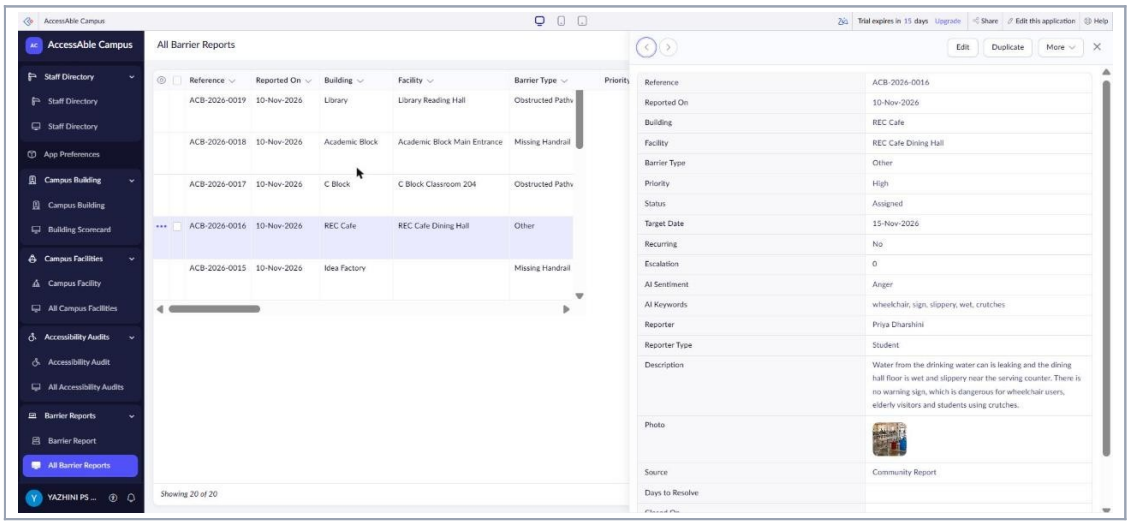


Figure 5.28: Barrier Report Detail View Showing AI Sentiment and AI Keywords

5.10 Pro-Code Extension

No pro-code extension was used. The requirements were met by forms, reports, pages and Deluge functions, and no heavy processing, custom widget or external service was needed. If the Accessibility Cell later wants to analyse the repair history, for example to predict which facilities will fail, a Zoho Catalyst function or a Creator REST API client could be added without changing the existing forms.

5.11 Mobile and Portal Access

The application was previewed in the phone layout of Zoho Creator. In this preview the Staff Directory form is shown as a list of cards, and the platform offers the Zoho Creator mobile app for iOS and Android with the same application. A mobile-specific layout, offline settings and a customer portal were not configured.

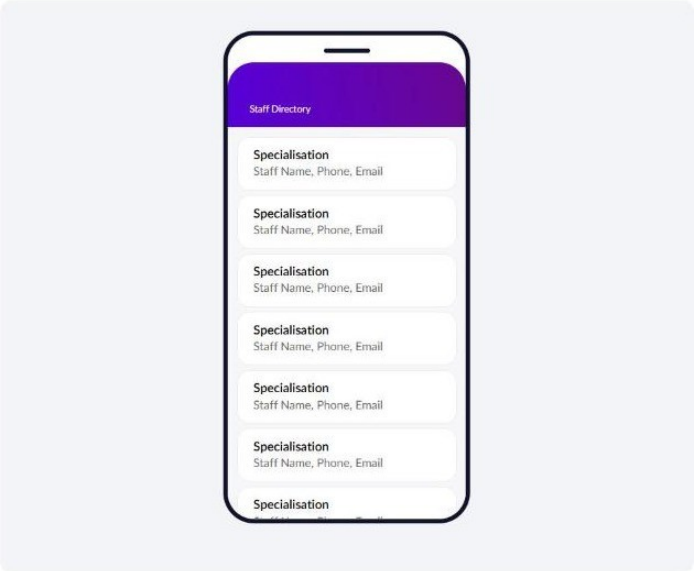


Figure 5.29: Application in the Phone Layout Preview of Zoho Creator

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done at four levels. **Unit testing** exercised each Deluge function on its own with the Execute option of the function editor, and checked the log messages and the resulting records. **Integration testing** followed a record through the chain of workflows: a barrier report was created, the function chain ran, and the status, corrective action, e-mail and Cliq messages were checked. **Data verification** compared the figures on the Command Centre with the record counts in the reports (12 buildings, 18 audits, 20 facilities, 20 barrier reports and 13 corrective actions). **Role-based and user-acceptance testing** need separate sign-ins for each role and are planned for the demonstration; they are marked as planned in Table 6.1 and Table 6.3, and no result has been claimed for them.

6.2 Test Cases

Table 6.1 lists twenty test cases that cover every functional requirement of Table 3.2. Status Pass means that the expected result was observed in the live application on 10 November 2026; Planned means that the case is to be executed in front of the examiner.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Mandatory fields on the Barrier Report form	Open the form in live mode	Building, Barrier Type and Description are marked mandatory	Asterisk shown on the three fields	Pass
TC-02	FR-01, NFR-03	Reporter sees only the fields needed	Open the form as a reporter	System fields are hidden	Ten reporter fields shown; reference, status, AI fields hidden	Pass
TC-03	FR-02	Reference number format	Open All Barrier Reports	References like ACB-2026-0016	All 20 records carry ACB-2026-NNNN	Pass
TC-04	FR-03	Audit score and result	Open All Accessibility Audits	Score and result follow the 80 / 50 limits	AUD-2026-0018 100 % Accessible; 0016 75 % Partially Accessible; 0012 37.5 % Not Accessible	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-05	FR-04	Building rating	Open Building Scorecard	Green ≥ 80, Amber ≥ 50, else Red	Library Block 96.5 Green; Academic Block 70.8 Amber; BB Court 33.3 Red	Pass
TC-06	FR-05	Recurring barrier detection	Three lift reports for Academic Block Lift A	Barrier flagged recurring	Command Centre shows 1 recurring barrier	Pass
TC-07	FR-06	Target date from priority	High priority barrier reported 10 Nov 2026	Target date 15 Nov 2026 (5 days)	ACT-2026-0016 target date 15-Nov-2026	Pass
TC-08	FR-07	AI keywords and sentiment stored	Open recent barrier reports	Emotion and accessibility words stored	AI Sentiment and AI Keywords columns filled (e.g. wheelchair, visually impaired)	Pass
TC-09	FR-08	Cliq alert for Critical / High	Submit Critical or High report	Message in #access-alerts	URGENT alert for ACB-2026-0018 visible in the channel	Pass
TC-10	FR-09	Approval creates corrective action	Set status Approved	Action created, assigned to the right team, status Assigned	ACT-2026-0016 assigned to Ravi Shankar (Civil Maintenance), Not Started	Pass
TC-11	FR-10, FR-11	Verified work closes the barrier	Verifier accepts completed action	Barrier Closed with days to resolve	ACT-2026-0008 verified; ACB-2026-0008 closed with resolution days	Pass
TC-12	FR-11	Resolution message	Close a barrier	Resolved message in Cliq	“Resolved: …” messages visible in the channel	Pass
TC-13	FR-12	Escalation digest	Run escalation for overdue barriers	ESCALATION digest in Cliq	Digest listing three overdue barriers with levels posted	Pass
TC-14	FR-12	Schedule enabled	Open Workflow → Schedules	Daily 08:30 schedule is enabled	Daily Overdue Escalation: Enabled	Pass
TC-15	FR-13	Audit raises barriers automatically	Submit audit with a Does Not Meet item	Barrier created once, no duplicate	To be run in the demonstration	Planned
TC-16	FR-14	Dashboard figures equal record counts	Compare Command Centre with reports	Counts agree	12 buildings, 18 audits, 20 facilities, 20 barriers agree	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-17	FR-15	Kanban, calendar and pivot reports	Open the three reports	Records displayed by status, date and building	All three open and show records	Pass
TC-18	NFR-02, FR-16	Staff cannot open Command Centre	Sign in as an Auditor-profile user	Access denied	To be run in the demonstration	Planned
TC-19	FR-01	Submit without Building	Leave Building empty and submit	Error message; no record	To be run in the demonstration	Planned
TC-20	NFR-05	Sample loader is idempotent	Execute loadSampleData again	No duplicate data	Log: “Data already loaded – skipped”	Pass

Of the twenty cases, seventeen passed and three are planned for the demonstration.

6.3 Debugging Deluge Scripts

Four techniques were used. The info statement writes values to the execution log, for example the response of every Cliq call. The Execute option of the function editor runs a function on its own and shows a result and log messages (Figure 6.1). try-catch blocks around the Cliq and Zia calls store the outcome in the Cliq_Status and AI_Sentiment fields, so a failed call is visible in the record instead of being lost. Finally, the error message shown by Creator on a failed submission was read to locate the line.

One real bug is described in Table 6.2 as BUG-01. When the first alert was posted, the Cliq call returned the message “Connection access is disabled for this application”. The log made it clear that the problem was in the connection and not in the script, and the connection was then re-authorised and enabled for the application.

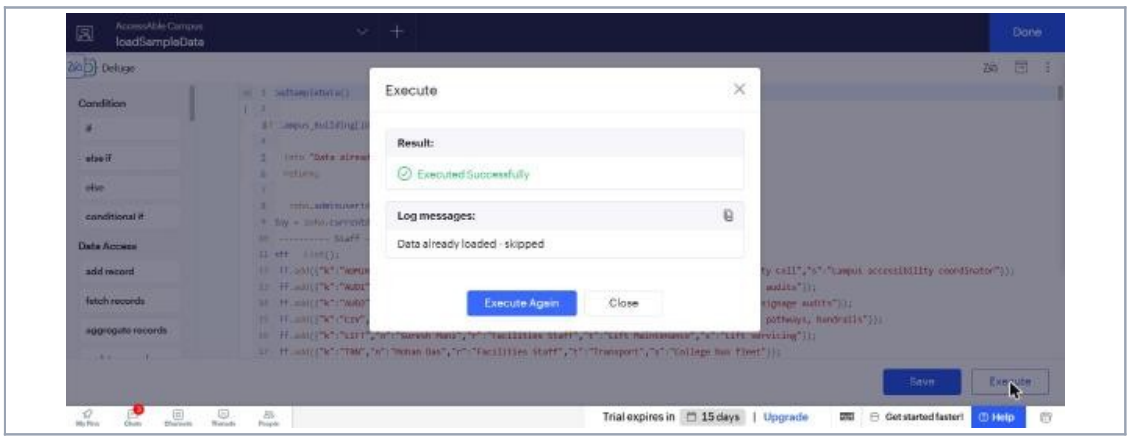


Figure 6.1: Deluge Function Execution Result and Log Message

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Cliq post failed with “Connection access is disabled for this application”	The Cliq connection was not enabled for use by the application scripts	Connection re-authorised and enabled; Cliq_Status now records Posted or Failed	Closed
BUG-02	Sentiment analysis failed for long descriptions	The Zia function rejects text longer than its limit	Text cut to 190 characters and the call wrapped in try-catch with the value Unavailable	Closed
BUG-03	Daily escalation did not run after the application was imported	The schedule came in disabled	Schedule enabled and verified (Figure 5.23)	Closed
BUG-04	Editing an already closed barrier re-sent the closure e-mail and Cliq message	WF-03 runs on every edit and the closure branch has no “already notified” check	Known; planned fix is to run the closure branch only when Closed_On is empty	Open

6.5 User Acceptance Testing

Feedback from end users has not been collected at the time of writing, and no feedback has therefore been recorded. Table 6.3 is the acceptance plan for the demonstration, in which each role performs one task in the live application.

Table 6.3: User Acceptance Testing Plan

Role to be Tested	Task	Acceptance Condition	Status
Community Member	Submit a barrier report on the Report and Track page	Reference number received by e-mail; status visible	Planned
Auditor	Submit an accessibility audit	Score, result and rating calculated automatically	Planned
Accessibility Administrator	Approve a report	Corrective action created and assigned	Planned
Facilities Staff / Verifier	Complete and verify the action	Barrier closed and closure certificate sent	Planned

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator provides Development, Stage and Production environments. AccessAble Campus was built and tested in the Development environment, which is the environment that the live URL serves. The Environments page shows that no application has been added to Stage or Production yet (Figure 7.1), so the application has not been published to Production. The deployment plan is to add the application to Stage for a final test with the roles, to publish it to Production with version notes, and to keep the previous version for rollback. Table 7.1 records the status of each checklist item honestly.

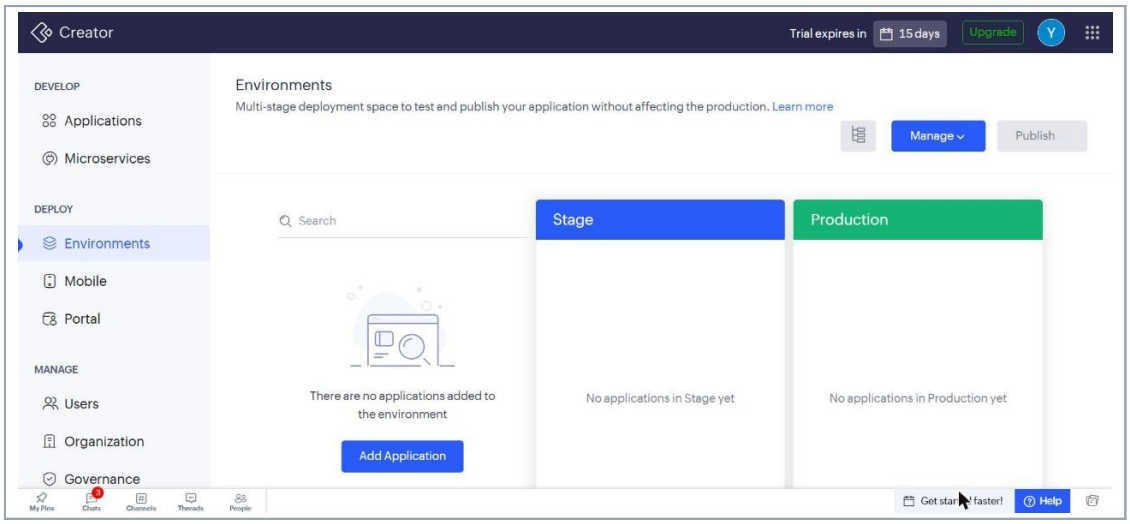


Figure 7.1: Environments Page (Stage and Production Empty)

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓ 17 of 20; 3 planned
2	Sample/test data removed or isolated	✗ retained for the demonstration (marked as sample data)
3	Roles, permissions and users configured	✓ profiles defined; users to be invited
4	Connections authorised	✓ Cliq connection (Development)
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓ live URL opens (Development); ✗ Production not yet published

S.No.	Checklist Item	Done (✓)
7	Version / release notes recorded	✗ not yet recorded

7.2 Performance Optimisation

No timing measurements were taken, because the data set is small. The optimisations listed in Table 7.2 are design choices that reduce the number of records read by a function, and the figures are record counts per execution.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before (naive approach)	After (implemented)	Improvement
Latest audit of each facility read with sort by date and range from 1 to 1	DS-06 refreshBuilding	Read every audit of the facility	Reads 1 audit per facility	Constant cost as audit history grows
Overdue barriers selected by criteria in the fetch statement	DS-07 escalateOverdue	Read all barrier reports and test each in a loop	Reads only open barriers past their target date (2 actions overdue on the current data)	Fewer records processed each morning
Recurrence and duplicate checks use count with criteria	DS-01, DS-04	Fetch records and count in a loop	One aggregate count	No loop over records
Single orchestration function per event	WF-01 to WF-04	Several actions per workflow	One function call per workflow	Rules in one place; easy to change

7.3 Security Features

Security rests on the platform and on a few design decisions (Table 7.3). Encrypted fields, IP restrictions and two-factor authentication were not configured in this version, and the Security Policy of the Governance console has not been set up (Figure 7.2).

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Permission profiles	Settings → User Permissions (Auditor and Facilities Staff; Community Member)	Unauthorised access to forms, reports and pages
OAuth 2.0 connection	zoho_cliq_connection used by the Cliq calls	Credential leakage: no key or password in code
Hidden system fields	On Load workflows of Barrier Report and Accessibility Audit	Tampering with reference, status, dates and AI results by reporters
Mandatory fields	Building, Barrier Type, Description and others	Incomplete or invalid data

Security Control	Where Applied	Threat Mitigated
try-catch around external calls	Cliq and Zia calls	Failure of an external service blocking a report
Sign-in through Zoho accounts	Application access	Anonymous use of the application

7.4 Governance and Administration

The platform keeps an activity history of changes to records and to the application. Naming conventions are applied throughout: references use the prefixes ACB-, ACT- and AUD- with the year and a four-digit number, link names use underscores, and workflows and scripts are numbered WF-, SCH- and DS-. The application can be exported with the Application IDE for backup, and users and licences are managed from the Users section of the Creator workspace; the trial edition expires 15 days after creation. The Accessibility Administrator is responsible for the picklists, the staff directory, the permission profiles and the schedule, and the developer for the Deluge functions.

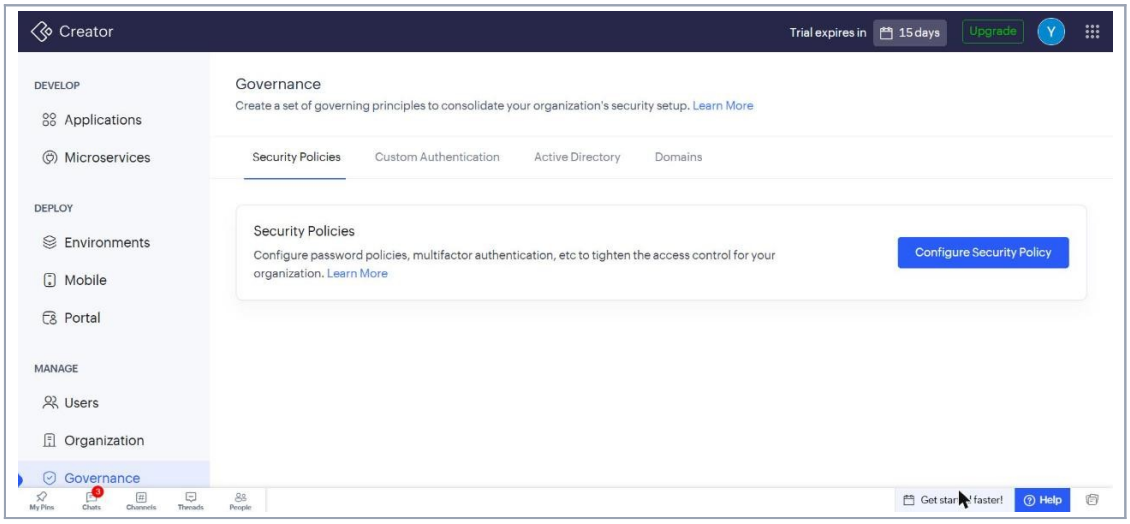


Figure 7.2: Governance Console (Security Policies Not Yet Configured)

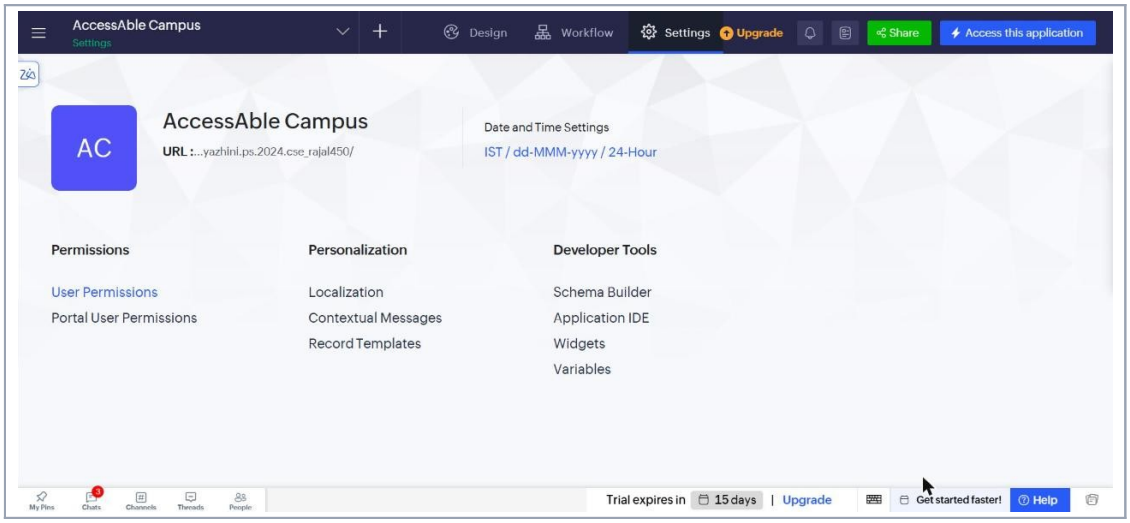


Figure 7.3: Application Settings with Permissions and Developer Tools

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario below follows a barrier through each role. A community member opens the Report and Track page, which shows the counters and the report form (Figure 8.1). On submission, the barrier receives a reference number, a priority-based target date and the AI fields (Figure 8.2). When the Administrator approves it, a corrective action is created and assigned to the matching team (Figure 8.3). The staff member completes the work and the verifier accepts it (Figure 8.4), after which the barrier is closed with its resolution days (Figure 8.5). The screenshots show the live records ACB-2026-0016 and ACT-2026-0016 for the start of the process and ACT-2026-0008 and ACB-2026-0008 for its completion.

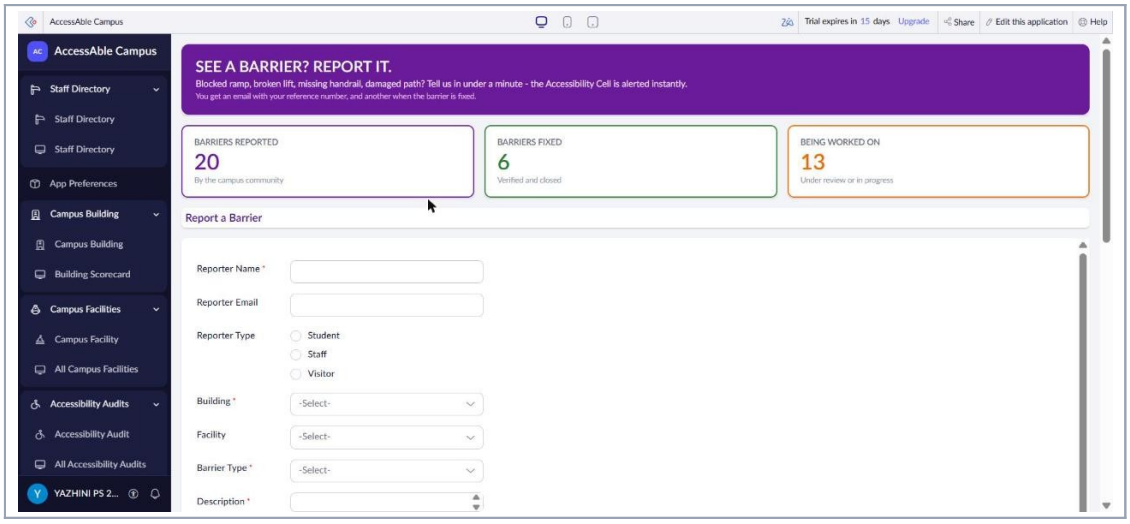


Figure 8.1: Scenario Step 1 – Community Member Opens Report and Track

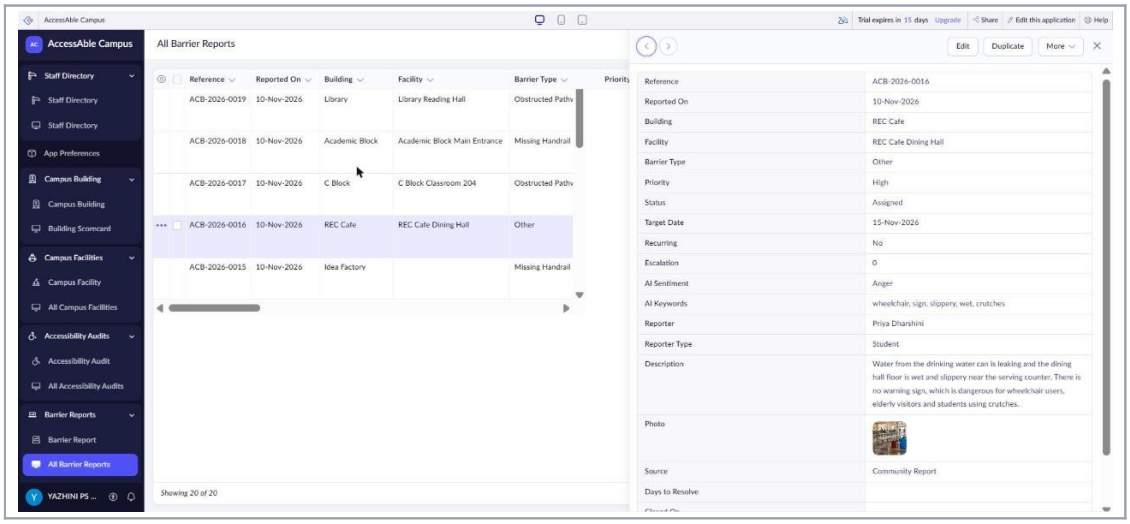


Figure 8.2: Scenario Step 2 – Barrier Report Stored with Reference, Target Date and AI Fields

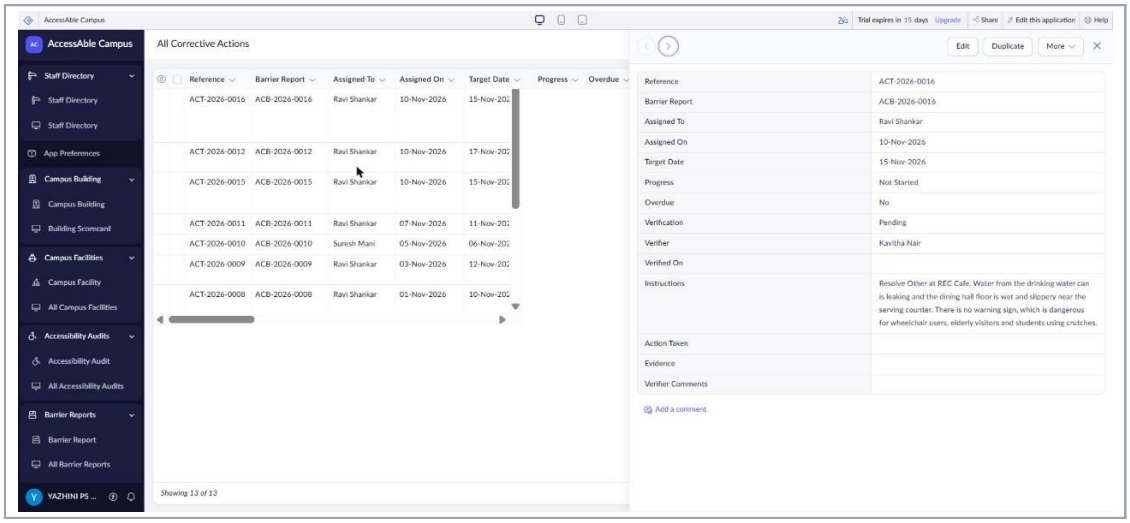


Figure 8.3: Scenario Step 3 – Corrective Action Created and Assigned

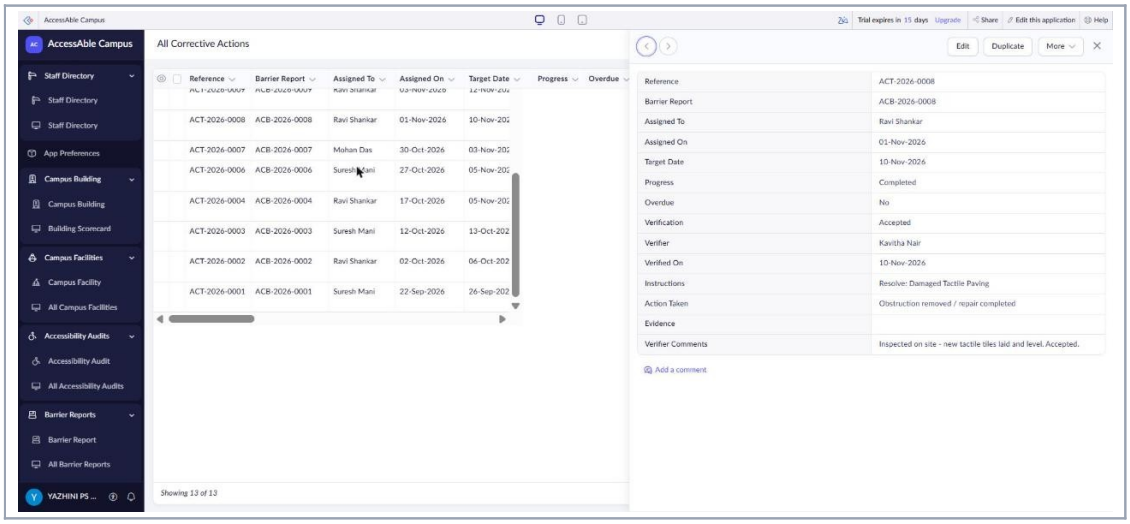


Figure 8.4: Scenario Step 4 – Completed Action Verified by the Verifier

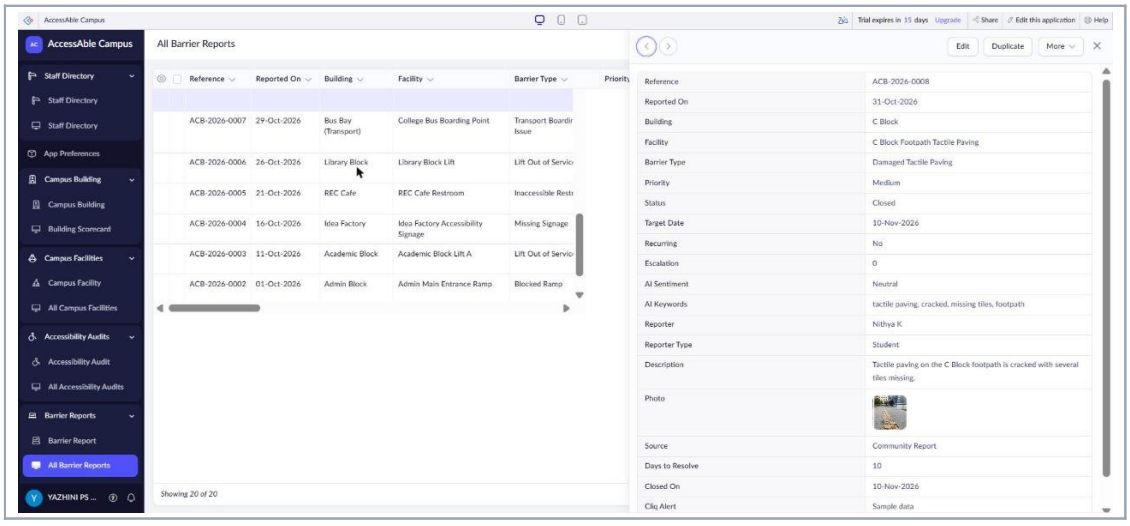


Figure 8.5: Scenario Step 5 – Barrier Closed with Days to Resolve

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Normalised data model and forms	Sections 4.2, 4.3, 5.2	Six related forms; Figure 5.6 (Schema Builder)
Automate scoring, numbering, recurrence, target dates, routing, escalation and certificates	Sections 5.5, 5.6	Eight functions and seven workflows; test cases TC-04 to TC-14
Role-based dashboards and reports	Sections 5.3, 5.4, 5.8	Nine reports and three pages; permission profiles
Integrate with Cliq, e-mail and Zia	Sections 5.7, 5.9	Cliq messages and AI fields visible in the application
Organise access and document testing, deployment and governance	Chapters 6 and 7	Test cases, defect log, deployment checklist (honest status)

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Depends on conversations and follow-up visits; no deadline	Reference number, status and target date set automatically on submission	Every request has a visible deadline (2 to 20 days by priority)
Manual steps	Noting, forwarding, following up and re-checking by hand	Numbering, routing, alerts, e-mail and escalation are automatic	Manual numbering, routing and reminders removed
Report generation	Counted manually from registers	Command Centre, scorecard and pivot are calculated from live records	Instant, always current
Data accessibility	Paper or one person’s spreadsheet	Web and phone layout, role-based	Available to every authorised user

8.4 Limitations of the Low-Code Approach

- **Trial edition.** The 15-day trial limits how long the application can be kept without a subscription, and Stage and Production were not used.
- **Platform limits.** Deluge has execution limits, so each function does one job and the daily escalation reads only the records that matter. The Zia function rejected long text, which was handled by truncation (BUG-02).

- **Integration set-up.** The Cliq connection had to be enabled for the application before scripts could use it (BUG-01), and a schedule came in disabled (BUG-03).
- **Page design.** Pages are built from the platform's components, so the layout of the Command Centre follows the available widgets and a pixel-exact design was not possible.
- **Event model.** A workflow runs on every edit, so editing a closed barrier re-ran the closure logic (BUG-04); a guard condition is needed.
- **Vendor dependence.** The application depends on Zoho for hosting, the connection and the AI service.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of the application is no-code configuration: the six forms, nine reports, three pages, the lookups, the permission profiles and the connection need no code. Low-code Deluge was needed for the rules that a form cannot express, namely scoring, recurrence, routing, escalation, notification and the dashboard figures. Pro-code would be needed only if the cell wanted features outside the platform, such as machine-learning prediction of failures, a custom mobile app with offline photo capture or integration with building sensors. At that point a Zoho Catalyst function or an external service could be called through the Creator REST API, and the current functions would remain.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

AccessAble Campus replaces informal, verbal and register-based handling of campus accessibility problems with one Zoho Creator application. Six linked forms, nine reports and three pages hold and present the data; eight Deluge functions, six form workflows and a daily schedule score every audit, rate every building, number and prioritise every report, detect recurring barriers, route the repair, escalate overdue work and close it only after verification. Cliq alerts, e-mail and the Zia sentiment service keep the right people informed.

The application was tested with 12 buildings, 20 facilities, 18 audits and 20 barrier reports; seventeen of twenty test cases passed and three are planned for the demonstration. The work shows that a low-code platform lets a student build a complete, working business process in a short time, while its limits, such as the trial edition and the event model, must be understood and handled.

9.2 Future Enhancements

- Publish the application through Stage to Production with version notes, and invite real users to the Auditor and Facilities Staff and Community Member profiles.
- Guard the closure branch so that editing a closed barrier does not repeat notifications (BUG-04).
- Analyse the repair history and repeated failures with Zoho Analytics or Zia to predict which facilities need maintenance.
- Add QR codes on each facility so that a reporter can open the form with the building and facility already selected, and use the Zoho Creator mobile app for photographs and geo-tagging.
- Add an approval process or a Blueprint for the review and verification stages, and a customer portal for visitors.

9.3 Learning Outcomes and Career Relevance

Through this project the student learned data modelling with lookup fields, form and report design, building dashboard pages, writing Deluge functions with try-catch, criteria fetches and integration tasks, configuring form and schedule workflows, connecting Zoho Cliq with OAuth, using the Zia AI service and planning testing and deployment. Debugging the Cliq connection and the long-text limit taught how to

design for failure. These skills suit for the roles of citizen developer, low-code developer, solutions consultant and automation engineer.

REFERENCES

[1] W. Rosam and O. Kalnichevski, Low-Code Development: A Practical Guide. New Delhi, India: BPB Publications.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 10 Nov. 2026).

[3] A. C. Bock and U. Frank, “Low-Code Platform,” Business & Information Systems Engineering, vol. 63, no. 6, 2021.

[4] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020.

[5] Y. Luo, P. Liang, C. Wang, M. Shahin, and J. Zhan, “Characteristics and challenges of low-code development: The practitioners’ perspective,” in Proc. 15th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM), 2021.

[6] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 10 Nov. 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Only features that exist in the live application are marked Y. A feature that is only partly implemented is marked Partial and a feature that was not used is marked N, with the reason in Chapter 5, 7 or 8. Section numbers refer to this report.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	AccessAble Campus; left menu sections	5.1	Fig. 5.1, 5.2
II	Forms with ≥ 8 distinct field types	Y	Six forms (Barrier Report, Accessibility Audit, …)	5.2	Fig. 5.4, 5.5
II	Lookup field (relationship between forms)	Y	Building, Facility, Assigned_To, Barrier_Report, …	4.2, 5.2	Fig. 4.2, 5.3, 5.6
II	Subform (parent–child data)	N	Not used; checklist items are fixed drop-down fields	4.2	—
II	Field validations / layout rules	Partial	Mandatory fields; On Load hide rules (no custom validation script)	5.2	Fig. 5.4
II	List report with filter, grouping, custom action	Partial	Six list reports; no custom action buttons	5.3	Fig. 5.7
II	Kanban / Calendar / Timeline / Map report	Y	Barrier Workflow Board (kanban); Action Due Calendar	5.3	Fig. 5.8
II	Pivot table / chart report	Y	Building Barrier Pivot; Command Centre charts	5.3, 5.4	Fig. 5.12, 5.15
II	Page / dashboard built with UI Builder	Y	Command Centre, Staff Workboard, Report and Track	5.4	Fig. 5.15–5.18
II	Deluge – form events (validate / success / field actions)	Y	On Success and On Load workflows	5.5, 5.6	Code 5.1–5.4
II	Deluge – functions, fetch, loops, collections	Y	Eight functions	5.5	Code 5.1–5.4; App. C
II	Email / in-app notifications	Y	sendmail tasks; Cliq alerts	5.5, 5.6	Fig. 5.21
II	Form workflow	Y	WF-01 to WF-06	5.6	Fig. 5.21, 5.22
II	Scheduled workflow	Y	SCH-01 Daily Overdue Escalation	5.6	Fig. 5.23
II	Approval process	N	Approval done by status change, not an approval process	5.6	—
II	Blueprint	N	Not used	5.6	—

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	Test cases, defect log, execution log	6	Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Cliq channel access-alerts	5.7	Fig. 5.24, 5.25
III	Connection / REST API call (invokeurl)	Partial	OAuth connection used by Cliq task; no invokeurl call	5.7	Fig. 5.24
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Y	Auditor and Facilities Staff; Community Member	5.8	Fig. 5.27
III	Record-level access criteria / customer portal	N	Not configured	5.8	—
III	Zia / AI feature	Y	zoho.ai.analyseSentiment; keyword extraction	5.9	Code 5.2; Fig. 5.28
III	Analytics (charts / Zoho Analytics)	Y	Charts and pivot report (Zoho Analytics not used)	5.4, 5.9	Fig. 5.12, 5.15
IV	Performance optimisation applied & measured	Partial	Applied; counts compared, no timings	7.2	Table 7.2
IV	Deployment across environments	N	Development only; Stage and Production empty	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	OAuth and permission profiles; no encrypted fields	7.3	Table 7.3
IV	Governance (audit trail, administration)	Partial	Platform history and settings; security policy not set	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used; justified	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	Partial	Phone layout preview only	5.11	Fig. 5.29

Total features fully implemented: 17 / 32; partially implemented: 7; not used: 8.

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

Attainment levels: 1 – Low, 2 – Medium, 3 – High.

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The complete source of every Deluge script is given below, grouped by function or workflow, with the script ID, location and purpose in the header comment.

C.1 Workflows WF-01 to WF-04 (call the functions)

```
// Script ID : DS-01 to DS-04
// Location  : Form workflows on Accessibility_Audit,
Barrier_Report and Corrective_Action
// Purpose   : Call the functions that hold the logic

// ---- WF-01 On Audit Submitted | Accessibility_Audit | Created |
On Success ----
thisapp.handleAudit(input.ID);

// ---- WF-02 On Barrier Reported | Barrier_Report | Created | On
Success ----
thisapp.processBarrierReport(input.ID);
r = Barrier_Report[ID == input.ID];
if(r.Source == "Audit Finding" && r.Status == "Approved")
{
    thisapp.handleBarrierStatus(input.ID);
}

// ---- WF-03 On Barrier Status Change | Barrier_Report | Edited |
On Success ----
// Runs whenever an administrator updates a barrier (approve /
reject / close)
thisapp.handleBarrierStatus(input.ID);

// ---- WF-04 On Action Progress | Corrective_Action | Created or
Edited | On Success ----
// Syncs staff progress and verifier decision back to the barrier
report
thisapp.handleActionUpdate(input.ID);
```

C.2 Form On Load scripts WF-05 and WF-06

```
// Script ID : DS-09
// Location  : Barrier_Report and Accessibility_Audit → Created →
On Load
// Purpose   : Hide system fields; default audit date

// ---- WF-05 Report Form Load | Barrier_Report | Created | On Load
----
// Reporters only see the fields they need - system fields are
filled automatically
hide Report_Reference;
hide Status;
hide Source;
hide Reported_On;
hide Target_Date;
hide Is_Recurring;
hide Recurrence_Count;
hide AI_Keywords;
```

```
hide AI_Sentiment;
hide Escalation_Level;
hide Cliq_Status;
hide Closed_On;
hide Days_To_Resolve;

// ---- WF-06 Audit Form Load | Accessibility_Audit | Created | On
Load ----
// Score, result and reference are calculated automatically on
submit
input.Audit_Date = zoho.currentdate;
hide Audit_Reference;
hide Audit_Score;
hide Overall_Result;
hide Failed_Items;
hide Audit_Status;
```

C.3 Function processBarrierReport

```
// Script ID : DS-01
// Location  : Function; called by WF-02
// Purpose   : Reference, recurrence, SLA date, sentiment,
keywords, Cliq alert, acknowledgement e-mail

void processBarrierReport(int rId)
{
    r = Barrier_Report[ID == rId];
    if(r.count() == 0)
    {
        return;
    }
    today = zoho.currentdate;
    // 1. Reference number and dates
    if(isBlank(r.Report_Reference))
    {
        seq = Barrier_Report[ID != 0].count();
        r.Report_Reference="ACB-" + today.getYear() + "-" + ("0000"
+ seq).right(4);
    }
    if(r.Reported_On == null)
    {
        r.Reported_On=today;
    }
    if(isBlank(r.Priority))
    {
        r.Priority="Medium";
    }
    // 2. Recurring barrier detection (same facility/building +
same type in last 60 days)
    if(r.Facility != null)
    {
        similar = Barrier_Report[Facility == r.Facility &&
Barrier_Type == r.Barrier_Type && Reported_On >=
today.subDay(60)].count();
    }
    else
    {
```

```
        similar = Barrier_Report[Building == r.Building &&
Barrier_Type == r.Barrier_Type && Reported_On >=
today.subDay(60)].count();
    }
    r.Recurrence_Count=similar;
    if(similar >= 3)
    {
        r.Is_Recurring="Yes";
        if(r.Priority == "Low")
        {
            r.Priority="Medium";
        }
        else if(r.Priority == "Medium")
        {
            r.Priority="High";
        }
    }
    else
    {
        r.Is_Recurring="No";
    }
    // 3. Target date from priority (SLA)
    slaDays = {"Critical":2,"High":5,"Medium":10,"Low":20};
    r.Target_Date=r.Reported_On.addDay(slaDays.get(r.Priority));
    r.Escalation_Level=0;
    // 4. AI sentiment (Zia) - advisory only, never used to set
priority
    desc = ifnull(r.Description,"");
    if(desc != "")
    {
        try
        {
            sent = if(desc.length() >
190,desc.subString(0,190),desc);
            ai = zoho.ai.analyseSentiment(sent);
            emo = "";
            if(ai.containKey("data"))
            {
                emo = ai.get("data").get(0).get("Emotion");
            }
            else if(ai.containKey("Prediction"))
            {
                emo = ai.get("Prediction");
            }
            r.AI_Sentiment=ifnull(emo,"Unavailable");
        }
        catch (e)
        {
            r.AI_Sentiment="Unavailable";
        }
        // 5. Keyword extraction (accessibility vocabulary match)
        vocab =
{"wheelchair","ramp","lift","elevator","stairs","steps","handrail",
"railing","blocked","obstructed","parked","bike","construction","po
thole","crack","broken","damaged","tactile","signage","sign","restr
oom","toilet","door","narrow","slippery","wet","spill","dark","ligh
ting","bus","boarding","crutches","blind","visually
```

```
impaired","hearing","entrance","pathway","path","parking","stuck","
trapped","out of order","not working"};
        low = desc.toLowerCase();
        kw = List();
        for each v in vocab
        {
            if(low.contains(v))
            {
                kw.add(v);
            }
        }
        r.AI_Keywords=if(kw.size() > 0,kw.toString(", "),"(none
detected)");
    }
    // 6. Cliq alert for Critical / High / Recurring
    bName = ifnull(Campus_Building[ID ==
r.Building].Building_Name,"Campus");
    fName = "";
    if(r.Facility != null)
    {
        fName = " / " + Campus_Facility[ID ==
r.Facility].Facility_Name;
    }
    if(r.Priority == "Critical" || r.Priority == "High" ||
r.Is_Recurring == "Yes")
    {
        msg = if(r.Priority == "Critical","URGENT - ","") +
"Accessibility barrier " + r.Report_Reference + " | " + r.Priority
+ " | " + r.Barrier_Type + " at " + bName + fName + " | Reported by
" + r.Reporter_Name + " | Due " + r.Target_Date.toString("dd-MMM-
yyyy");
        if(r.Is_Recurring == "Yes")
        {
            msg = msg + " | RECURRING (" + similar + " similar
reports in 60 days)";
        }
        msg = msg + " | " + desc;
        try
        {
            resp =
zoho.cliq.postToChannel("accessalerts",msg,"zoho_cliq_connection");
            info resp;
            r.Cliq_Status=if(resp.get("status") ==
"OK","Posted","Failed");
        }
        catch (e)
        {
            r.Cliq_Status="Failed";
        }
    }
    else
    {
        r.Cliq_Status="Not required (" + r.Priority + ")";
    }
    // 7. Email acknowledgement to reporter
    if(!isBlank(r.Reporter_Email))
    {
```

```
        sendmail
        [
            from :zoho.adminuserid
            to :r.Reporter_Email
            subject :"AccessAble Campus - Report " +
r.Report_Reference + " received"
            message :"<p>Dear " + r.Reporter_Name + ",</p><p>Thank
you for reporting an accessibility barrier. Your report has been
recorded.</p><p><b>Reference:</b> " + r.Report_Reference +
"<br><b>Location:</b> " + bName + fName + "<br><b>Barrier:</b> " +
r.Barrier_Type + "<br><b>Priority:</b> " + r.Priority +
"<br><b>Target resolution:</b> " + r.Target_Date.toString("dd-MMM-
yyyy") + "</p><p>The Accessibility Cell will review it and keep you
updated.</p><p>- AccessAble Campus</p>"
        ]
    }
    thisapp.refreshBuilding(r.Building);
}
```

C.4 Function handleBarrierStatus

```
// Script ID : DS-02
// Location  : Function; called by WF-02 and WF-03
// Purpose   : Create and route corrective action; rejection and
closure handling

void handleBarrierStatus(int rId)
{
    r = Barrier_Report[ID == rId];
    if(r.count() == 0)
    {
        return;
    }
    today = zoho.currentdate;
    bName = ifnull(Campus_Building[ID ==
r.Building].Building_Name,"Campus");
    st = r.Status;
    // Approved -> auto-create and auto-route a corrective action
to the right team
    if(st == "Approved")
    {
        if(Corrective_Action[Barrier_Report == rId].count() == 0)
        {
            team = "Civil Maintenance";
            if(r.Barrier_Type == "Lift Out of Service")
            {
                team = "Lift Maintenance";
            }
            else if(r.Barrier_Type == "Transport Boarding Issue")
            {
                team = "Transport";
            }
            else if(r.Barrier_Type == "Missing Signage" ||
r.Barrier_Type == "Inaccessible Restroom")
            {
                team = "Civil Maintenance";
            }
```

```
            stf = Staff_Directory[Team == team && Staff_Role ==
"Facilities Staff" && Staff_Status == "Active"] range from 1 to 1;
            if(stf.count() == 0)
            {
                stf = Staff_Directory[Staff_Role == "Facilities
Staff" && Staff_Status == "Active"] range from 1 to 1;
            }
            ver = Staff_Directory[Staff_Role == "Verifier" &&
Staff_Status == "Active"] range from 1 to 1;
            if(stf.count() > 0)
            {
                insert into Corrective_Action
                [
                    Added_User=zoho.loginuser

Action_Reference=r.Report_Reference.replaceAll("ACB-","ACT-")
                    Barrier_Report=rId
                    Assigned_To=stf.ID
                    Assigned_On=today
                    Target_Date=r.Target_Date
                    Work_Instructions="Resolve " + r.Barrier_Type +
" at " + bName + ". " + ifnull(r.Description,"")
                    Progress="Not Started"
                    Verifier=if(ver.count() > 0,ver.ID,null)
                    Verification_Result="Pending"
                    Is_Overdue="No"
                ]
                r.Status="Assigned";
                msg = "Assigned: " + r.Report_Reference + " (" +
r.Priority + " - " + r.Barrier_Type + " at " + bName + ") assigned
to " + stf.Staff_Name + " [" + team + "]. Due " +
r.Target_Date.toString("dd-MMM-yyyy");
                try
                {
                    resp =
zoho.cliq.postToChannel("accessalerts",msg,"zoho_cliq_connection");
                }
                catch (e)
                {
                    info e;
                }
                sendmail
                [
                    from :zoho.adminuserid
                    to :stf.Staff_Email
                    subject :"AccessAble Campus - New corrective
action " + r.Report_Reference
                    message :"<p>Dear " + stf.Staff_Name +
",</p><p>A new accessibility barrier has been assigned to
you.</p><p><b>Reference:</b> " + r.Report_Reference +
"<br><b>Barrier:</b> " + r.Barrier_Type + " at " + bName +
"<br><b>Priority:</b> " + r.Priority + "<br><b>Due:</b> " +
r.Target_Date.toString("dd-MMM-yyyy") + "</p><p>" +
ifnull(r.Description,"") + "</p><p>Please update progress in
AccessAble Campus.</p>"
                ]
            }
```

```
        }
    }
    // Rejected -> inform reporter
    if(st == "Rejected" && !isBlank(r.Reporter_Email))
    {
        sendmail
        [
            from :zoho.adminuserid
            to :r.Reporter_Email
            subject :"AccessAble Campus - Report " +
r.Report_Reference + " reviewed"
            message :"<p>Dear " + r.Reporter_Name + ",</p><p>Your
report " + r.Report_Reference + " was reviewed and closed without
action (duplicate or not an accessibility barrier). Please contact
the Accessibility Cell if the problem persists.</p>"
        ]
    }
    // Closed -> resolution metrics + closure email
    if(st == "Closed" || st == "Verified")
    {
        if(r.Closed_On == null)
        {
            r.Closed_On=today;
        }
        r.Days_To_Resolve=daysBetween(r.Reported_On,r.Closed_On);
        r.Status="Closed";
        if(!isBlank(r.Reporter_Email))
        {
            sendmail
            [
                from :zoho.adminuserid
                to :r.Reporter_Email
                subject :"Barrier Closure Certificate - " +
r.Report_Reference
                message :"<div style='max-
width:620px;margin:auto;border:6px double
#1E2A5A;padding:28px;font-family:Georgia,serif;text-
align:center'><div style='font-size:13px;letter-
spacing:3px;color:#5B6475'>ACCESSABLE CAMPUS</div><h1
style='color:#1E2A5A;margin:8px 0'>Barrier Closure
Certificate</h1><p style='font-size:15px'>This certifies that the
accessibility barrier reported by <b>" + r.Reporter_Name + "</b>
has been resolved and verified.</p><table style='margin:16px
auto;font-size:14px;text-align:left'><tr><td><b>Reference</b></td><
td>" + r.Report_Reference +
"</td></tr><tr><td><b>Barrier</b></td><td>" + r.Barrier_Type +
"</td></tr><tr><td><b>Location</b></td><td>" + bName +
"</td></tr><tr><td><b>Priority</b></td><td>" + r.Priority +
"</td></tr><tr><td><b>Reported on</b></td><td>" +
r.Reported_On.toString("dd-MMM-yyyy") +
"</td></tr><tr><td><b>Closed on</b></td><td>" +
r.Closed_On.toString("dd-MMM-yyyy") +
"</td></tr><tr><td><b>Resolved in</b></td><td>" + r.Days_To_Resolve
+ " day(s)</td></tr></table><p style='font-
size:13px;color:#5B6475'>Thank you for helping make our campus
accessible for everyone.</p><p style='font-
size:12px;color:#8A93A6'>Accessibility Cell - AccessAble
```

```
Campus</p></div>"
            ]
        }
        try
        {
            resp =
zoho.cliq.postToChannel("accessalerts","Resolved: " +
r.Report_Reference + " (" + r.Barrier_Type + " at " + bName + ")
closed in " + r.Days_To_Resolve + "
day(s).","zoho_cliq_connection");
        }
        catch (e)
        {
            info e;
        }
    }
    thisapp.refreshBuilding(r.Building);
}
```

C.5 Function handleActionUpdate

```
// Script ID : DS-03
// Location  : Function; called by WF-04
// Purpose   : Keep the barrier status in step with the corrective
action

void handleActionUpdate(int cId)
{
    c = Corrective_Action[ID == cId];
    if(c.count() == 0)
    {
        return;
    }
    r = Barrier_Report[ID == c.Barrier_Report];
    if(r.count() == 0)
    {
        return;
    }
    today = zoho.currentdate;
    if(isBlank(c.Action_Reference))
    {

c.Action_Reference=r.Report_Reference.replaceAll("ACB-","ACT-");
    }
    if(c.Assigned_On == null)
    {
        c.Assigned_On=today;
    }
    if(c.Target_Date == null)
    {
        c.Target_Date=r.Target_Date;
    }
    c.Is_Overdue=if(c.Progress != "Completed" && c.Target_Date !=
null && c.Target_Date < today,"Yes","No");
    bName = ifnull(Campus_Building[ID ==
r.Building].Building_Name,"Campus");
    if(c.Verification_Result == "Accepted")
```

```
    {
        if(c.Verified_On == null)
        {
            c.Verified_On=today;
        }
        c.Progress="Completed";
        c.Is_Overdue="No";
        r.Status="Verified";
        r.Closed_On=c.Verified_On;
        thisapp.handleBarrierStatus(r.ID);
    }
    else if(c.Verification_Result == "Rework Required")
    {
        r.Status="Rework Required";
        c.Progress="In Progress";
        c.Verification_Result="Pending";
        stf = Staff_Directory[ID == c.Assigned_To];
        try
        {
            resp = zoho.cliq.postToChannel("accessalerts","Rework
required: " + c.Action_Reference + " (" + r.Barrier_Type + " at " +
bName + ") was not accepted by the verifier. Comment: " +
ifnull(c.Verification_Comments,"-"),"zoho_cliq_connection");
        }
        catch (e)
        {
            info e;
        }
        if(stf.count() > 0)
        {
            sendmail
            [
                from :zoho.adminuserid
                to :stf.Staff_Email
                subject :"AccessAble Campus - Rework required on "
+ c.Action_Reference
                message :"<p>The verifier has asked for rework on "
+ c.Action_Reference + " (" + r.Barrier_Type + " at " + bName +
").</p><p><b>Comments:</b> " + ifnull(c.Verification_Comments,"-")
+ "</p>"
            ]
        }
    }
    else if(c.Progress == "Completed")
    {
        r.Status="Awaiting Verification";
        ver = Staff_Directory[ID == c.Verifier];
        try
        {
            resp = zoho.cliq.postToChannel("accessalerts","Ready
for verification: " + c.Action_Reference + " (" + r.Barrier_Type +
" at " + bName + "). Action taken: " +
ifnull(c.Action_Taken,"-"),"zoho_cliq_connection");
        }
        catch (e)
        {
            info e;
```

```
        }
        if(ver.count() > 0)
        {
            sendmail
            [
                from :zoho.adminuserid
                to :ver.Staff_Email
                subject :"AccessAble Campus - Please verify " +
c.Action_Reference
                message :"<p>Work on " + c.Action_Reference + " ("
+ r.Barrier_Type + " at " + bName + ") is marked
complete.</p><p><b>Action taken:</b> " + ifnull(c.Action_Taken,"-")
+ "</p><p>Please inspect and record the verification result.</p>"
            ]
        }
    }
    else if(c.Progress == "In Progress")
    {
        r.Status="Work in Progress";
    }
    else
    {
        if(r.Status == "Approved" || r.Status == "Under Review" ||
r.Status == "Submitted")
        {
            r.Status="Assigned";
        }
    }
    thisapp.refreshBuilding(r.Building);
}
```

C.6 Function handleAudit

```
// Script ID : DS-04
// Location  : Function; called by WF-01
// Purpose   : Raise barriers from audit findings; alert on poor
audits

void handleAudit(int aId)
{
    thisapp.computeAuditScore(aId);
    a = Accessibility_Audit[ID == aId];
    if(a.count() == 0)
    {
        return;
    }
    f = Campus_Facility[ID == a.Facility];
    if(f.count() == 0)
    {
        return;
    }
    aud = Staff_Directory[ID == a.Auditor];
    // map checklist item -> (value, barrier type, priority)
    items = List();
    items.add({"v":a.Ramp_Access,"t":"Blocked
Ramp","p":"High","l":"Ramp not available / obstructed"});
    items.add({"v":a.Pathway_Clear,"t":"Obstructed
```

```
Pathway","p":"Medium","l":"Pathway not clear or level"});
    items.add({"v":a.Signage_Present,"t":"Missing
Signage","p":"Low","l":"Accessibility signage missing"});
    items.add({"v":a.Lift_Operational,"t":"Lift Out of
Service","p":"High","l":"Lift not operational"});
    items.add({"v":a.Restroom_Accessible,"t":"Inaccessible
Restroom","p":"High","l":"Accessible restroom unavailable"});
    items.add({"v":a.Parking_Usable,"t":"Parking
Misuse","p":"Medium","l":"Accessible parking not usable"});
    items.add({"v":a.Handrails_Present,"t":"Missing
Handrail","p":"High","l":"Handrails missing on stairs/ramp"});
    items.add({"v":a.Tactile_Paving,"t":"Damaged Tactile
Paving","p":"Medium","l":"Tactile paving damaged"});
    created = 0;
    for each it in items
    {
        if(it.get("v") == "Does Not Meet")
        {
            existing = Barrier_Report[Facility == f.ID &&
Barrier_Type == it.get("t") && Status != "Closed" && Status !=
"Rejected"].count();
            if(existing == 0)
            {
                nid = insert into Barrier_Report
                [
                    Added_User=zoho.loginuser
                    Reporter_Name=ifnull(aud.Staff_Name,"Auditor")

Reporter_Email=ifnull(aud.Staff_Email,zoho.adminuserid)
                    Reporter_Type="Staff"
                    Building=f.Campus_Building
                    Facility=f.ID
                    Barrier_Type=it.get("t")
                    Description="Audit finding (" +
ifnull(a.Audit_Reference,"audit") + "): " + it.get("l") + " at " +
f.Facility_Name + ". " + ifnull(a.Observations,"")
                    Priority=it.get("p")
                    Status="Approved"
                    Source="Audit Finding"
                    Reported_On=a.Audit_Date
                ];
                thisapp.processBarrierReport(nid);
                thisapp.handleBarrierStatus(nid);
                created = created + 1;
            }
        }
    }
    // Notify reviewer on Cliq if facility is poorly rated
    if(ifnull(a.Audit_Score,100) < 50 || created > 0)
    {
        msg = "Audit " + a.Audit_Reference + " submitted for " +
f.Facility_Name + " | Score " + a.Audit_Score + "% (" +
a.Overall_Result + ") | " + created + " new barrier(s) raised
automatically. Please review.";
        try
        {
            resp =
```

```
zoho.cliq.postToChannel("accessalerts",msg,"zoho_cliq_connection");
            info resp;
        }
        catch (e)
        {
            info e;
        }
    }
    a.Audit_Status="Submitted";
}
```

C.7 Function computeAuditScore

```
// Script ID : DS-05
// Location  : Function; called by DS-04 and DS-08
// Purpose   : Audit score, result, reference and next inspection
date

void computeAuditScore(int aId)
{
    a = Accessibility_Audit[ID == aId];
    if(a.count() > 0)
    {
        vals = List();
        vals.add(ifnull(a.Ramp_Access,"N/A"));
        vals.add(ifnull(a.Pathway_Clear,"N/A"));
        vals.add(ifnull(a.Signage_Present,"N/A"));
        vals.add(ifnull(a.Lift_Operational,"N/A"));
        vals.add(ifnull(a.Restroom_Accessible,"N/A"));
        vals.add(ifnull(a.Parking_Usable,"N/A"));
        vals.add(ifnull(a.Handrails_Present,"N/A"));
        vals.add(ifnull(a.Tactile_Paving,"N/A"));
        pts = 0.0;
        applicable = 0;
        failed = 0;
        for each v in vals
        {
            if(v == "Meets")
            {
                pts = pts + 1;
                applicable = applicable + 1;
            }
            else if(v == "Needs Improvement")
            {
                pts = pts + 0.5;
                applicable = applicable + 1;
            }
            else if(v == "Does Not Meet")
            {
                applicable = applicable + 1;
                failed = failed + 1;
            }
        }
        score = 0.0;
        if(applicable > 0)
        {
            score = (pts / applicable * 100).round(1);
```

```
        }
        a.Audit_Score=score;
        a.Failed_Items=failed;
        a.Overall_Result=if(score >= 80,"Accessible",if(score >=
50,"Partially Accessible","Not Accessible"));
        if(isBlank(a.Audit_Reference))
        {
            seq = Accessibility_Audit[ID <= aId].count();
            a.Audit_Reference="AUD-" + a.Audit_Date.getYear() + "-"
+ ("0000" + seq).right(4);
        }
        f = Campus_Facility[ID == a.Facility];
        if(f.count() > 0)
        {
            f.Next_Inspection_Due=a.Audit_Date.addDay(if(score >=
80,180,if(score >= 50,90,30)));
            thisapp.refreshBuilding(f.Campus_Building);
        }
    }
}
```

C.8 Function refreshBuilding

```
// Script ID : DS-06
// Location  : Function; called by several scripts
// Purpose   : Open barriers, building score and rating

void refreshBuilding(int bId)
{
    b = Campus_Building[ID == bId];
    if(b.count() > 0)
    {
        openCount = Barrier_Report[Building == bId && Status !=
"Closed" && Status != "Rejected"].count();
        total = 0.0;
        cnt = 0;
        lastDate = '01-Jan-2000';
        for each f in Campus_Facility[Campus_Building == bId]
        {
            la = Accessibility_Audit[Facility == f.ID] sort by
Audit_Date desc range from 1 to 1;
            if(la.count() > 0)
            {
                if(la.Audit_Score != null)
                {
                    total = total + la.Audit_Score;
                    cnt = cnt + 1;
                    if(la.Audit_Date > lastDate)
                    {
                        lastDate = la.Audit_Date;
                    }
                }
            }
        }
        b.Open_Barriers=openCount;
        if(cnt > 0)
        {
```

```
            score = (total / cnt).round(1);
            b.Accessibility_Score=score;
            b.Last_Audit_Date=lastDate;
            b.Rating=if(score >= 80,"Green",if(score >=
50,"Amber","Red"));
        }
        else
        {
            b.Rating="Not Audited";
        }
    }
}
```

C.9 Function escalateOverdue

```
// Script ID : DS-07
// Location  : Function; called by SCH-01
// Purpose   : Escalate overdue barriers; digest by Cliq and e-mail

void escalateOverdue()
{
    today = zoho.currentdate;
    lines = List();
    cnt = 0;
    for each r in Barrier_Report[Status != "Closed" && Status !=
"Rejected" && Target_Date < today]
    {
        lvl = ifnull(r.Escalation_Level,0) + 1;
        r.Escalation_Level=lvl;
        daysLate = daysBetween(r.Target_Date,today);
        bName = ifnull(Campus_Building[ID ==
r.Building].Building_Name,"Campus");
        lines.add(r.Report_Reference + " - " + r.Barrier_Type + "
at " + bName + " (" + r.Priority + ", " + daysLate + " day(s)
overdue, level " + lvl + ")");
        cnt = cnt + 1;
        for each c in Corrective_Action[Barrier_Report == r.ID &&
Progress != "Completed"]
        {
            c.Is_Overdue="Yes";
        }
    }
    for each c2 in Corrective_Action[Progress == "Completed"]
    {
        c2.Is_Overdue="No";
    }
    // Facilities due for re-inspection
    dueF = Campus_Facility[Next_Inspection_Due <=
today.addDay(7)].count();
    if(cnt > 0)
    {
        msg = "ESCALATION - " + cnt + " accessibility barrier(s)
are past their target date:\n- " + lines.toString("\n- ");
        if(dueF > 0)
        {
            msg = msg + "\n" + dueF + " facility/facilities due for
re-inspection within 7 days.";
```

```
        }
        try
        {
            resp =
zoho.cliq.postToChannel("accessalerts",msg,"zoho_cliq_connection");
            info resp;
        }
        catch (e)
        {
            info e;
        }
        sendmail
        [
            from :zoho.adminuserid
            to :zoho.adminuserid
            subject :"AccessAble Campus - Daily escalation: " + cnt
+ " overdue barrier(s)"
            message :"<p>The following accessibility barriers are
overdue and have been escalated:</p><ul><li>" +
lines.toString("</li><li>") + "</li></ul><p>" + dueF + "
facility/facilities are due for re-inspection within 7 days.</p>"
        ]
    }
    for each b in Campus_Building[ID != 0]
    {
        thisapp.refreshBuilding(b.ID);
    }
    info cnt + " overdue barrier(s) escalated";
}
```

C.10 Schedule SCH-01

```
// Script ID : DS-07
// Location  : Schedule – daily at 08:30
// Purpose   : Run the escalation function

// ---- SCH-01 Daily Overdue Escalation | Schedule | Daily 08:30
IST (from 31-Oct-2026) ----
// Every day at 08:30 - escalate overdue barriers to Cliq + email
and refresh building scores
thisapp.escalateOverdue();
```

C.11 Function loadSampleData

```
// Script ID : DS-08
// Location  : Function; run manually
// Purpose   : Load sample data when the application is empty

void loadSampleData()
{
    if(Campus_Building[ID != 0].count() > 0)
    {
        info "Data already loaded - skipped";
        return;
    }
    em = zoho.adminuserid;
    today = zoho.currentdate;
```

```
    // ---------- Staff ----------
    staff = List();
    staff.add({"k":"ADMIN","n":"Dr. Meena
Raghavan","r":"Accessibility Administrator","t":"Accessibility
Cell","s":"Campus accessibility coordinator"});
    staff.add({"k":"AUD1","n":"Arun
Kumar","r":"Auditor","t":"Accessibility Cell","s":"Building access
audits"});
    staff.add({"k":"AUD2","n":"Priya
Sundar","r":"Auditor","t":"Accessibility Cell","s":"Restroom and
signage audits"});
    staff.add({"k":"CIV","n":"Ravi Shankar","r":"Facilities
Staff","t":"Civil Maintenance","s":"Ramps, pathways, handrails"});
    staff.add({"k":"LIFT","n":"Suresh Mani","r":"Facilities
Staff","t":"Lift Maintenance","s":"Lift servicing"});
    staff.add({"k":"TRN","n":"Mohan Das","r":"Facilities
Staff","t":"Transport","s":"College bus fleet"});
    staff.add({"k":"VER","n":"Kavitha
Nair","r":"Verifier","t":"Accessibility Cell","s":"Work
verification"});
    sId = Map();
    for each s in staff
    {
        nid = insert into Staff_Directory
        [
            Added_User=zoho.loginuser
            Staff_Name=s.get("n")
            Staff_Email=em
            Staff_Role=s.get("r")
            Team=s.get("t")
            Specialisation=s.get("s")
            Staff_Status="Active"
        ];
        sId.put(s.get("k"),nid);
    }
    // ---------- Buildings ----------
    blds = List();
    blds.add({"c":"ADM","n":"Admin
Block","t":"Administrative","f":3});
    blds.add({"c":"WSB","n":"Workshop
Block","t":"Academic","f":2});
    blds.add({"c":"LIB","n":"Library","t":"Library","f":2});
    blds.add({"c":"HUT","n":"Hut Cafe","t":"Cafeteria","f":1});
    blds.add({"c":"RCF","n":"REC Cafe","t":"Cafeteria","f":1});
    blds.add({"c":"IDF","n":"Idea
Factory","t":"Innovation","f":2});
    blds.add({"c":"TLG","n":"Tech Lounge","t":"Recreation","f":1});
    blds.add({"c":"CBL","n":"C Block","t":"Academic","f":4});
    blds.add({"c":"ACB","n":"Academic
Block","t":"Academic","f":5});
    blds.add({"c":"LBB","n":"Library Block","t":"Library","f":3});
    blds.add({"c":"BBC","n":"BB Court","t":"Sports","f":0});
    blds.add({"c":"BUS","n":"Bus Bay
(Transport)","t":"Transport","f":0});
    bId = Map();
    for each b in blds
    {
```

```
        nid = insert into Campus_Building
        [
            Added_User=zoho.loginuser
            Building_Name=b.get("n")
            Building_Code=b.get("c")
            Building_Type=b.get("t")
            Floors=b.get("f")
            Rating="Not Audited"
            Open_Barriers=0
            Building_Status="Active"
        ];
        bId.put(b.get("c"),nid);
    }
    // ---------- Facilities ----------
    facs = List();
    facs.add({"k":"ADM-RAMP","b":"ADM","n":"Admin Main Entrance
Ramp","t":"Ramp","fl":"Ground"});
    facs.add({"k":"ADM-LIFT","b":"ADM","n":"Admin Block
Lift","t":"Lift","fl":"Ground"});
    facs.add({"k":"ACB-LIFTA","b":"ACB","n":"Academic Block Lift
A","t":"Lift","fl":"Ground"});
    facs.add({"k":"ACB-RR","b":"ACB","n":"Academic Block Accessible
Restroom","t":"Restroom","fl":"Ground"});
    facs.add({"k":"ACB-ENT","b":"ACB","n":"Academic Block Main
Entrance","t":"Entrance","fl":"Ground"});
    facs.add({"k":"LIB-STEPS","b":"LIB","n":"Library Front
Steps","t":"Staircase","fl":"Ground"});
    facs.add({"k":"LBB-LIFT","b":"LBB","n":"Library Block
Lift","t":"Lift","fl":"Ground"});
    facs.add({"k":"LBB-PARK","b":"LBB","n":"Library Block
Accessible Parking","t":"Parking","fl":"Outdoor"});
    facs.add({"k":"WSB-PATH","b":"WSB","n":"Workshop Access
Pathway","t":"Pathway","fl":"Outdoor"});
    facs.add({"k":"HUT-ENT","b":"HUT","n":"Hut Cafe
Entrance","t":"Entrance","fl":"Ground"});
    facs.add({"k":"RCF-RR","b":"RCF","n":"REC Cafe
Restroom","t":"Restroom","fl":"Ground"});
    facs.add({"k":"IDF-SIGN","b":"IDF","n":"Idea Factory
Accessibility Signage","t":"Signage","fl":"Ground"});
    facs.add({"k":"TLG-RAMP","b":"TLG","n":"Tech Lounge
Ramp","t":"Ramp","fl":"Ground"});
    facs.add({"k":"CBL-TACT","b":"CBL","n":"C Block Footpath
Tactile Paving","t":"Tactile Paving","fl":"Outdoor"});
    facs.add({"k":"CBL-STAIR","b":"CBL","n":"C Block Main
Staircase","t":"Staircase","fl":"All floors"});
    facs.add({"k":"BBC-PATH","b":"BBC","n":"BB Court Access
Path","t":"Pathway","fl":"Outdoor"});
    facs.add({"k":"BUS-BOARD","b":"BUS","n":"College Bus Boarding
Point","t":"Bus Boarding","fl":"Outdoor"});
    fId = Map();
    for each f in facs
    {
        nid = insert into Campus_Facility
        [
            Added_User=zoho.loginuser
            Facility_Name=f.get("n")
            Campus_Building=bId.get(f.get("b"))
```

```
            Facility_Type=f.get("t")
            Floor=f.get("fl")
            Facility_Status="Operational"
        ];
        fId.put(f.get("k"),nid);
    }
    // ---------- Audits ----------
    cm = {"M":"Meets","N":"Needs Improvement","D":"Does Not
Meet","X":"N/A"};
    auds = List();
    auds.add({"f":"ACB-ENT","d":80,"a":"AUD1","c":"NNMXXMNM"});
    auds.add({"f":"ADM-RAMP","d":75,"a":"AUD1","c":"DMNXXMNM"});
    auds.add({"f":"ACB-LIFTA","d":70,"a":"AUD2","c":"MNMDXXMN"});
    auds.add({"f":"BBC-PATH","d":65,"a":"AUD1","c":"XDNXXXXN"});
    auds.add({"f":"HUT-ENT","d":60,"a":"AUD2","c":"DNDXXXNX"});
    auds.add({"f":"LIB-STEPS","d":55,"a":"AUD1","c":"XMNXXXDN"});
    auds.add({"f":"CBL-TACT","d":50,"a":"AUD2","c":"XNMXXXMD"});
    auds.add({"f":"RCF-RR","d":45,"a":"AUD2","c":"XNNXDXNX"});
    auds.add({"f":"ACB-RR","d":40,"a":"AUD1","c":"XMNXDXMX"});
    auds.add({"f":"LBB-LIFT","d":35,"a":"AUD2","c":"MMMNXMMM"});
    auds.add({"f":"IDF-SIGN","d":30,"a":"AUD1","c":"MMDXXXMM"});
    auds.add({"f":"WSB-PATH","d":28,"a":"AUD2","c":"XDNXXXNN"});
    auds.add({"f":"BUS-BOARD","d":25,"a":"AUD1","c":"XXNXXXDX"});
    auds.add({"f":"ADM-RAMP","d":20,"a":"AUD2","c":"MMMXXMMN"});
    auds.add({"f":"TLG-RAMP","d":18,"a":"AUD1","c":"NMMXXXMM"});
    auds.add({"f":"ACB-LIFTA","d":12,"a":"AUD2","c":"MMNDXXMM"});
    auds.add({"f":"CBL-STAIR","d":9,"a":"AUD1","c":"XMMXXXNM"});
    auds.add({"f":"LBB-PARK","d":5,"a":"AUD2","c":"MMMXXMMM"});
    for each a in auds
    {
        c = a.get("c");
        nid = insert into Accessibility_Audit
        [
            Added_User=zoho.loginuser
            Facility=fId.get(a.get("f"))
            Auditor=sId.get(a.get("a"))
            Audit_Date=today.subDay(a.get("d"))
            Ramp_Access=cm.get(c.subString(0,1))
            Pathway_Clear=cm.get(c.subString(1,2))
            Signage_Present=cm.get(c.subString(2,3))
            Lift_Operational=cm.get(c.subString(3,4))
            Restroom_Accessible=cm.get(c.subString(4,5))
            Parking_Usable=cm.get(c.subString(5,6))
            Handrails_Present=cm.get(c.subString(6,7))
            Tactile_Paving=cm.get(c.subString(7,8))
            Observations="Routine accessibility inspection"
            Audit_Status="Reviewed"
        ];
        thisapp.computeAuditScore(nid);
    }
    // ---------- Barrier reports ----------
    pDays = {"Critical":2,"High":5,"Medium":10,"Low":20};
    brs = List();
    brs.add({"b":"ACB","f":"ACB-LIFTA","t":"Lift Out of
Service","p":"High","s":"Closed","d":50,"cd":47,"rn":"Karthik
R","rt":"Student","st":"Disappointment","kw":"lift, not responding,
stairs","w":"LIFT","desc":"Lift A in the Academic Block is not
```

```
responding and students with mobility needs are forced to use the stairs."});
    brs.add({"b":"ADM","f":"ADM-RAMP","t":"Blocked Ramp","p":"High","s":"Closed","d":40,"cd":36,"rn":"Anitha S","rt":"Staff","st":"Anger","kw":"wheelchair ramp, blocked, motorbikes, entrance","w":"CIV","desc":"The wheelchair ramp at the Admin Block entrance is blocked by parked motorbikes during office hours."});
    brs.add({"b":"ACB","f":"ACB-LIFTA","t":"Lift Out of Service","p":"Critical","s":"Closed","d":30,"cd":28,"rn":"Divya P","rt":"Student","st":"Anger","kw":"lift, stopped, between floors","w":"LIFT","desc":"Lift A stopped working again between floors with two students inside for ten minutes."});
    brs.add({"b":"IDF","f":"IDF-SIGN","t":"Missing Signage","p":"Low","s":"Closed","d":25,"cd":15,"rn":"Visitor - Ramesh","rt":"Visitor","st":"Neutral","kw":"accessibility sign, faded, peeled","w":"CIV","desc":"The accessibility sign near the Idea Factory entrance is faded and partly peeled off."});
    brs.add({"b":"RCF","f":"RCF-RR","t":"Inaccessible Restroom","p":"High","s":"Rejected","d":20,"cd":-1,"rn":"Sneha M","rt":"Student","st":"Disappointment","kw":"restroom, narrow door, REC Cafe","w":"","desc":"No accessible restroom near REC Cafe, the door is too narrow for a wheelchair. (Duplicate of an existing audit finding)"});
    brs.add({"b":"LBB","f":"LBB-LIFT","t":"Lift Out of Service","p":"Medium","s":"Rework Required","d":15,"cd":-1,"rn":"Prof. Lakshmi","rt":"Staff","st":"Disappointment","kw":"lift buttons, floor 2, not working","w":"LIFT","desc":"Library Block lift call buttons on floor 2 are not working."});
    brs.add({"b":"BUS","f":"BUS-BOARD","t":"Transport Boarding Issue","p":"High","s":"Work in Progress","d":12,"cd":-1,"rn":"Arjun V","rt":"Student","st":"Disappointment","kw":"bus steps, high, grab rail, boarding","w":"TRN","desc":"College bus entrance steps are very high and there is no grab rail, making boarding difficult for students with crutches."});
    brs.add({"b":"CBL","f":"CBL-TACT","t":"Damaged Tactile Paving","p":"Medium","s":"Awaiting Verification","d":10,"cd":-1,"rn":"Nithya K","rt":"Student","st":"Neutral","kw":"tactile paving, cracked, missing tiles, footpath","w":"CIV","desc":"Tactile paving on the C Block footpath is cracked with several tiles missing."});
    brs.add({"b":"TLG","f":"TLG-RAMP","t":"Blocked Ramp","p":"Medium","s":"Closed","d":8,"cd":5,"rn":"Harish B","rt":"Student","st":"Anger","kw":"ramp, blocked, stacked chairs, Tech Lounge","w":"CIV","desc":"The ramp at the Tech Lounge is blocked by stacked chairs after an event."});
    brs.add({"b":"ACB","f":"ACB-LIFTA","t":"Lift Out of Service","p":"Critical","s":"Work in Progress","d":6,"cd":-1,"rn":"Meera J","rt":"Student","st":"Anger","kw":"Lift A, out of service, no notice","w":"LIFT","desc":"Lift A near the staircase has been out of service for three days with no notice board."});
    brs.add({"b":"LIB","f":"LIB-STEPS","t":"Missing Handrail","p":"High","s":"Assigned","d":4,"cd":-1,"rn":"Gokul S","rt":"Student","st":"Disappointment","kw":"library steps, no handrail, crutches","w":"CIV","desc":"Library entrance steps have no handrail, which is hard for students using crutches."});
```

```
    brs.add({"b":"BBC","f":"BBC-PATH","t":"Uneven
Surface","p":"Medium","s":"Approved","d":3,"cd":-1,"rn":"Coach
Vinod","rt":"Staff","st":"Neutral","kw":"pothole, path, BB
court","w":"","desc":"The path to the BB court has a large pothole
that a wheelchair cannot cross."});
    brs.add({"b":"WSB","f":"WSB-PATH","t":"Obstructed
Pathway","p":"Medium","s":"Under Review","d":2,"cd":-1,"rn":"Sanjay
T","rt":"Student","st":"Disappointment","kw":"pathway, sand bags,
bricks, workshop","w":"","desc":"Construction sand bags and bricks
are blocking the pathway to the Workshop Block."});
    brs.add({"b":"HUT","f":"HUT-ENT","t":"Uneven
Surface","p":"Medium","s":"Submitted","d":1,"cd":-1,"rn":"Fathima
Z","rt":"Student","st":"Disappointment","kw":"entrance, raised
step, tables, wheelchair","w":"","desc":"Hut Cafe entrance has a
raised step and tables placed so a wheelchair cannot pass."});
    seq = 0;
    liftCount = 0;
    for each r in brs
    {
        seq = seq + 1;
        rep = today.subDay(r.get("d"));
        tgt = rep.addDay(pDays.get(r.get("p")));
        st = r.get("s");
        closedOn = null;
        dtr = null;
        if(r.get("cd") >= 0)
        {
            closedOn = today.subDay(r.get("cd"));
            dtr = r.get("d") - r.get("cd");
        }
        recurring = "No";
        rc = 0;
        if(r.get("f") == "ACB-LIFTA")
        {
            liftCount = liftCount + 1;
            rc = liftCount;
            if(liftCount >= 3)
            {
                recurring = "Yes";
            }
        }
        esc = 0;
        if(st != "Closed" && st != "Rejected" && tgt < today)
        {
            esc = 1;
        }
        rid = insert into Barrier_Report
        [
            Added_User=zoho.loginuser
            Report_Reference="ACB-" + rep.getYear() + "-" + ("0000"
+ seq).right(4)
            Reporter_Name=r.get("rn")
            Reporter_Email=em
            Reporter_Type=r.get("rt")
            Building=bId.get(r.get("b"))
            Facility=fId.get(r.get("f"))
            Barrier_Type=r.get("t")
```

```
            Description=r.get("desc")
            Priority=r.get("p")
            Status=st
            Source="Community Report"
            Reported_On=rep
            Target_Date=tgt
            Is_Recurring=recurring
            Recurrence_Count=rc
            AI_Keywords=r.get("kw")
            AI_Sentiment=r.get("st")
            Escalation_Level=esc
            Cliq_Status="Sample data"
            Closed_On=closedOn
            Days_To_Resolve=dtr
        ];
        w = r.get("w");
        if(w != "")
        {
            prog = "In Progress";
            vres = "Pending";
            verOn = null;
            if(st == "Assigned")
            {
                prog = "Not Started";
            }
            if(st == "Awaiting Verification")
            {
                prog = "Completed";
            }
            if(st == "Rework Required")
            {
                prog = "Completed";
                vres = "Rework Required";
            }
            if(st == "Closed")
            {
                prog = "Completed";
                vres = "Accepted";
                verOn = closedOn;
            }
            overdue = if(esc > 0,"Yes","No");
            insert into Corrective_Action
            [
                Added_User=zoho.loginuser
                Action_Reference="ACT-" + rep.getYear() + "-" +
("0000" + seq).right(4)
                Barrier_Report=rid
                Assigned_To=sId.get(w)
                Assigned_On=rep.addDay(1)
                Target_Date=tgt
                Work_Instructions="Resolve: " + r.get("t")
                Progress=prog
                Action_Taken=if(prog == "Completed","Obstruction
removed / repair completed","")
                Verifier=sId.get("VER")
                Verification_Result=vres
                Verified_On=verOn
```

```
                Is_Overdue=overdue
            ]
        }
    }
    for each bb in Campus_Building[ID != 0]
    {
        thisapp.refreshBuilding(bb.ID);
    }
    info "Sample data loaded";
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Open https://creatorapp.zoho.in/yazhini.ps.2024.cse_rajal450/accessable-campus/ in Chrome, Edge or Firefox.
2. Sign in with the Zoho account that the Administrator has invited to the application.
3. On a phone, install the Zoho Creator app from the App Store or Google Play, sign in with the same account and open AccessAble Campus.
4. The Command Centre opens first for the Administrator; other users use the menu on the left.

D.2 Instructions for the Community Member

1. Open Report and Track from the left menu.
2. Enter your name, e-mail and reporter type; select the building and facility.
3. Select the barrier type, describe the problem and attach a photograph if you have one.
4. Choose a priority and select Submit. A reference number is sent to your e-mail.
5. Open Barrier Reports → All Barrier Reports to follow the status using your reference.

D.3 Instructions for the Auditor

1. Open Accessibility Audits → Accessibility Audit.
2. Select the facility and your name; the date is filled with today.
3. Mark each of the eight items as Meets, Needs Improvement, Does Not Meet or N/A; add observations.
4. Select Submit. The score, result and next inspection date are calculated, and any Does Not Meet item raises a barrier automatically.
5. See the results in All Accessibility Audits and Building Scorecard.

D.4 Instructions for the Accessibility Administrator

1. Open Command Centre to see the key figures, charts and the priority action queue.
2. Open Barrier Reports → Barrier Workflow Board, or All Barrier Reports, and open a new report.

3. Edit the report and set the status to Approved to create and assign a corrective action, or to Rejected with a reason.
4. Watch overdue work on the Command Centre and in the Cliq channel; the daily digest arrives at 08:30.
5. Maintain buildings, facilities and staff through Campus Building, Campus Facilities and Staff Directory.

D.5 Instructions for Facilities Staff and the Verifier

1. Open Staff Workboard to see the actions assigned to you and their due dates.
2. Open Corrective Actions → All Corrective Actions, edit your action and set Progress to In Progress, then Completed, with the action taken.
3. The verifier opens the completed action, sets the Verification Result to Accepted or Rework Required and adds comments.
4. On acceptance the barrier is closed and the reporter receives a closure certificate by e-mail.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	YAZHINI PS	**Register No.**	2116240701612
Project Title	AccessAble Campus: Accessibility Audit and Barrier Resolution System	**Date**	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.

Level	% of Max. Marks	Descriptor
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner Name: _____	**Signature of External Examiner** Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	YAZHINI PS	**Register No.**	2116240701612
Department / Section	Computer Science and Engineering	**Date of Viva**	
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER Name: _____	**EXTERNAL EXAMINER** Name: _____